Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
Câmpus Birigui

Henrique José de Souza

Sistema Especialista Educacional para Validação de Compatibilidade de Hardware baseado em Problema de Satisfação de Restrições (CSP).

Birigui
2026

Henrique José de Souza

Sistema Especialista Educacional para Validação de Compatibilidade de Hardware baseado em Problema de Satisfação de Restrições (CSP).

Trabalho de Conclusão de Curso apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, como requisito parcial para conclusão do curso de Engenharia da Computação.

Área de Concentração: Área de Concentração do Trabalho

Orientador: Profa. Dra. Helen de Freitas Santos

Birigui

2026

Dedicatória (Opcional). Não digite a palavra Dedicatória. Texto no qual o autor do trabalho oferece homenagem ou dedica o seu trabalho a alguém (não usar ponto final)

AGRADECIMENTOS

Agradecimento (opcional). Folha que contém manifestação de reconhecimento a pessoas e/ou instituições que realmente contribuíram com o autor, devendo ser expressos de maneira simples.

Epígrafe (Opcional). Pensamentos retirados de um livro, uma música, um poema, normalmente relacionado ao tema do trabalho, seguida de indicação de autoria. As epígrafes podem ser colocadas também nas folhas de abertura de cada capítulo.

"*Any fool can write code that a computer can understand. Good programmers write code that humans can understand*".

Martin Fowler

RESUMO

Elemento obrigatório, constituído de uma sequência de frases concisas e objetivas, fornecendo uma visão rápida e clara do conteúdo do estudo. O texto deverá conter entre 150 a 250 palavras e ser antecedido pela referência do estudo. Também, não deve conter citações e deverá ressaltar o objetivo, o método, os resultados e as conclusões. O resumo deve ser redigido em parágrafo único, seguido das palavras representativas do conteúdo do estudo, isto é, palavras-chave, em número de três a cinco, separadas entre si por ponto e finalizadas também por ponto. Usar o verbo na terceira pessoa do singular, com linguagem impessoal (pronome SE), bem como fazer uso, preferencialmente, da voz ativa.

Palavras-chave: Palavra-chave 1. Palavra-chave 2. Palavra-chave 3. Palavra-chave n.

ABSTRACT OU RÉSUMÉ OU RESUMEN

Elemento obrigatório. É a versão do resumo em português para o idioma de divulgação internacional. Deve ser antecedido pela referência do estudo.

Keywords: Keyword 1. Keyword 2. Keyword 3. Keyword n.

Elemento obrigatório. É a versão do resumo em português para o idioma de divulgação internacional. Deve ser antecedido pela referência do estudo.

Mots-clés : Mot 1. Mot 2. Mot 3. Mot n.

Elemento obrigatório. É a versão do resumo em português para o idioma de divulgação internacional. Deve ser antecedido pela referência do estudo.

Palabras clave: Palabra 1. Palabra 2. Palabra 3. Palabra n.

LISTA DE ILUSTRAÇÕES

Figura 1 – Grafo de Restrições 23
Figura 2 – Execução do AC-3 27
Figura 3 – Diagrama da arquitetura em três camadas do sistema 33
Figura 4 – Diagrama de Casos de Uso 62
Figura 5 – Modelo de Domínio 65
Figura 6 – Diagrama de Atividades 67
Figura 7 – Diagrama de Estados 67
Figura 8 – Diagrama de Sequência 71
Figura 9 – Diferentes Detalhamentos dos Serviços Relacionados ao Cliente 72
Figura 10 – Exemplo de Caso de Teste Elaborado na Ferramenta Testlink 76

LISTA DE QUADROS

Quadro 1 – RF-01 – Listar componentes por categoria 45
Quadro 2 – RF-02 – Navegar pela interface *wizard* 46
Quadro 3 – RF-03 – Exibir componentes incompatíveis com sinalização visual . . 46
Quadro 4 – RF-04 – Exibir especificações técnicas dos componentes 47
Quadro 5 – RF-05 – Executar propagação de restrições via algoritmo AC-3 47
Quadro 6 – RF-06 – Validar compatibilidade de soquete CPU–Placa-mãe 48
Quadro 7 – RF-07 – Validar padrão de memória RAM–Placa-mãe 48
Quadro 8 – RF-08 – Validar capacidade energética da fonte (TDP) 49
Quadro 9 – RF-09 – Propagar restrições incrementalmente a cada seleção 49
Quadro 10 – RF-10 – Exibir justificativa técnica para cada incompatibilidade . . . 50
Quadro 11 – RF-11 – Exibir alerta de incompatibilidade de soquete 50
Quadro 12 – RF-12 – Exibir alerta de incompatibilidade de padrão de memória . . 51
Quadro 13 – RF-13 – Exibir alerta de fonte de alimentação subdimensionada 51
Quadro 14 – RF-14 – Reinicializar a configuração 52
Quadro 15 – RF-15 – Substituir componente selecionado em etapa anterior 52
Quadro 16 – RF-16 – Exibir resumo da configuração finalizada 53
Quadro 17 – RF-17 – Armazenar e consultar base de conhecimento do CSP 53
Quadro 18 – RF-18 – Popular base de conhecimento via *Database Seeding* 54
Quadro 19 – RNF-01 – Tempo de resposta da validação 54
Quadro 20 – RNF-02 – Ausência de avaliação por força bruta 55
Quadro 21 – RNF-03 – Linguagem natural nas explicações de incompatibilidade . . 55
Quadro 22 – RNF-04 – Interface responsiva 56
Quadro 23 – RNF-05 – Acesso sem cadastro ou autenticação 56
Quadro 24 – RNF-06 – Separação estrita entre as três camadas arquiteturais 57
Quadro 25 – RNF-07 – Motor de inferência expansível para novas restrições 57
Quadro 26 – RNF-08 – Base de conhecimento expansível para novas categorias . . . 58
Quadro 27 – RNF-09 – Restrições derivadas de documentação oficial dos fabricantes 58
Quadro 28 – RNF-10 – Determinismo do motor de inferência 59
Quadro 29 – RNF-11 – API *stateless* (sem estado de sessão no servidor) 59
Quadro 30 – RNF-12 – Ambiente de banco de dados reproduzível via Docker 60
Quadro 31 – RNF-13 – Rastreabilidade de cada bloqueio até a restrição violada . . 60
Quadro 32 – RNF-14 – Ausência de bloqueios silenciosos 61

LISTA DE TABELAS

Tabela 1 – Modelo para Especificação de Casos de Uso 63
Tabela 2 – Convenção para Nome dos Objetos no Banco de Dados 69
Tabela 3 – Tabelas Identificadas neste Trabalho 69
Tabela 4 – Estado 69
Tabela 5 – Cidade 70
Tabela 6 – NomeDaTabelaNN 70

LISTA DE ABREVIATURAS E SIGLAS

1D	Uma dimensão
2D	Duas dimensões
3D	Três dimensões

LISTA DE SÍMBOLOS

α	Letra grega minúscula Alfa
β	Letra grega minúscula Beta

SUMÁRIO

1 INTRODUÇÃO . . . 15
1.1 Justificativa . . . 16
1.2 Objetivos . . . 17
1.2.1 Objetivo Geral . . . 17
1.2.2 Objetivos Específicos . . . 17
1.3 Problema (Hipótese ou Questão de Pesquisa) . . . 17
1.4 Organização Deste Trabalho . . . 17

2 REVISÃO DA LITERATURA . . . 18
2.1 Compatibilidade de Hardware como Domínio do Problema . . . 18
2.1.1 Compatibilidade de Soquete CPU-Placa-mãe . . . 18
2.1.2 Padrão de Memória RAM . . . 18
2.1.3 Capacidade Energética e TDP . . . 19
2.1.4 Formalização como Problema de Restrições . . . 19
2.2 Problemas de Satisfação de Restrições (CSP) . . . 19
2.2.1 Tipos de Restrições . . . 20
2.2.2 Aplicação ao Domínio de Hardware . . . 20
2.2.3 Representação em Grafo . . . 21
2.3 Teoria dos Grafos e Grafo de Restrições . . . 21
2.3.1 Definições Fundamentais . . . 21
2.3.2 Representações Computacionais . . . 21
2.3.3 Grafo de Restrições . . . 22
2.4 Complexidade Computacional . . . 23
2.4.1 Implicações para o Domínio de Hardware . . . 24
2.5 Algoritmos de Busca e Backtracking . . . 24
2.5.1 Busca em Espaço de Atribuições . . . 24
2.5.2 Backtracking . . . 25
2.5.3 Forward Checking . . . 26
2.6 Propagação de Restrições e Algoritmo AC-3 . . . 26
2.6.1 Aplicação ao Domínio de Hardware . . . 26
2.7 Sistema Especialista . . . 28
2.7.1 Explanation Facilities . . . 28
2.7.2 Inteligência Artificial Explicável (XAI) . . . 29
2.7.3 Mapeamento ao Sistema Proposto . . . 29
2.8 Dimensão Educativa em Sistemas Especialistas . . . 30

2.8.1 Mecanismo de Filtragem versus Sistema com Explicação 30
2.8.2 Sistemas Tutores Inteligentes 30
2.8.3 Sistema Especialista Educacional 31
2.9 Arquitetura de Software Orientada a Serviços 31
2.9.1 Arquitetura em Camadas 31
2.9.2 REST como Estilo Arquitetural 32
2.9.3 Mapeamento ao Sistema Proposto 33
2.10 Trabalhos Relacionados 34
2.10.1 Plataformas Comerciais de Compatibilidade de Hardware 34
2.10.2 Trabalhos Acadêmicos Relacionados 34
2.10.3 Posicionamento do Trabalho Proposto 35

3 MATERIAIS E MÉTODOS 37
3.0.1 Bibliográfica 37
3.0.2 Aplicada 38
3.1 Materiais 38
3.1.1 Equipamentos 38
3.1.2 Softwares 38
3.2 Ferramentas Utilizadas 39
3.2.1 Visual Studio Code 39
3.2.2 Node.js e NestJS 39
3.2.3 PostgreSQL e Docker 39
3.2.4 DBeaver 40
3.2.5 Next.js 40
3.2.6 Figma 40
3.2.7 Astah Community 41
3.2.8 Teste? 41
3.3 Métodos de Desenvolvimento 41
3.3.1 Modelagem do Problema como CSP 41
3.3.2 Implementação do Algoritmo AC-3 41
3.3.3 Implementação das Justificativas Educativas 42
3.3.4 Processo de Desenvolvimento de Software 42
3.3.5 Estratégia de Testes 43

4 DESCRIÇÃO GERAL DO PRODUTO 44

5 ELICITAÇÃO DE REQUISITOS E ANÁLISE 45
5.1 Requisitos do Usuário 45
5.2 Requisitos do Sistema 45
5.2.1 Requisitos Funcionais 45

5.2.2 Requisitos Não Funcionais 53
5.2.3 Restrições, Suposições e Dependências 60
5.2.4 Requisitos Adiados 61
5.3 Casos de Uso 61
5.4 Casos de Uso 61
5.4.1 Diagrama de Casos de Uso 61
5.4.2 Especificação dos Casos de Uso 62
5.5 Modelo de Domínio 63
5.6 Diagrama de Objetos 66
5.7 Diagrama de Classes de Análise 66
5.8 Diagrama de Atividades 66
5.9 Diagrama de Estados 67

6 PROJETO DE SOFTWARE 68
6.1 Projeto de Interface 68
6.2 Projeto de Dados 68
6.2.1 Mapeamento Objeto-Relacional 68
6.2.2 Estrutura das Tabelas no Banco de Dados 68
6.2.3 Diagrama de Pacotes 70
6.2.4 Diagrama de Classes de Projeto 70
6.3 Projeto Procedimental 70
6.3.1 Diagrama de Sequência 70
6.3.2 Diagrama de Atividades 71
6.3.3 Diagrama de Estados 71
6.4 Projeto Arquitetural 71

7 IMPLEMENTAÇÃO 73

8 TESTE 74

9 IMPLANTAÇÃO 77

10 RESULTADOS E DISCUSSÃO 78

11 CONCLUSÕES/CONCLUSÕES PARCIAIS 79

12 CRONOGRAMA 80

Referências 81

1 INTRODUÇÃO

A montagem de um computador *desktop* exige o cruzamento simultâneo de especificações técnicas entre componentes interdependentes. Diferentemente da aquisição de periféricos genéticos, a configuração de um sistema computacional envolve restrições físicas, elétricas e lógicas que, quando violadas, resultam em incompatibilidade absoluta, os componentes adquiridos de forma incorreta simplesmente não funcionam junots, independentemente de ajustes de *software* ou configuração. A compatibilidade de soquete entre processador e placa-mãe, o padrão de memória suportado pela plataforma e o dimensionamento energético da fonte de alimentação são exemplos de restrições binárias e determinísticas que não admitem adaptações (Advanced Micro Devices, 2022; Intel Corporation, 2023; JEDEC Solid State Technology Association, 2020). Esse cenário impõe ao usuário um desafio técnico considerável, uma vez que a violão de qualquer dessas restrições representa prejuízo financeiro imediato e frustrante.

Diante dessa realidade, plataformas como o PC Part Picker e o Meupc.net se estabeleceram como referências globais e nacionais para a verificação de compatibilidade entre componentes. Ambas detectam automaticamente conflitos entre os itens selecionaos e alertam o usuário quando uma incompatibilidade é identificada (PCPartPicker LLC, 2024; Meupc.net, 2024). No entanto, quando o conflito é detectado, a resposta do sistema se limita a sinalizar que os componentes não são compatíveis entre si, sem indicar a regra física que foi violada. O usuário aprende o que não pode fazer, mas não aprende o motibo, e portanto, não está em condições de generalizar esse conhecimento para outras situações, como a escolha de um processador diferente ou a adição de um módulo de memória RAM.

Essa limitação não é acidental, ela decorre da arquitetura dessas plataformas que operam como mecanismos de filtragem e não como sistemas especialistas com capacidade de explicação. Segundo Clancey (1983), a capacidade de justificar o próprio raciocínio é o que distingue fundamentalmente um sistema especialista de um mecanismo de filtragem simples. Um filtro responde apenas à pergunta *"quais opções são válidas?"*, enquanto um sistema especialista dotado de *explanation facility* responde também à pergunta *"por que as demais são inválidas?"*, articulando a cadeia de regras e restrições que levou aquela conclusão. Nesse sentido, cada bloqueio silencioso de um mecanismo de filtragem é uma oportunidade de aprendizagem perdida ao usuário.

Do ponto de vista da ciência da computação, o problema de configuração de *hardware* recai formalmente sobre a classe dos Problemas de Satisfação de Restrições (CSP). Nessa formalização, cada categoria de componente corresponde a uma variável, o conjunto de modelos comercialmente disponíveis constitui o domínio dessa variável e

as regras físicas de compatibilidade entre pares de componentes definem as restrições do problema (Russell; Norvig, 2010). O CSP é classificado como NP-completo (Sipser, 2012), o que torna a avaliação exaustiva de todas as combinções inviável para catálogos de tamanho realista. Técnicas de propagação de restrições, como o algoritmo AC-3 proposto por Mackworth (1977), contornam essa explosão combinatória reduzindo o espaço de busca de forma incremental a cada interação do usuário, tornando a validação viável em milissegundos.

Neste contexto, este Trabalho de Conclusão de Curso tem como objetivo central abordar a lacuna identificada, desenvolvendo um sistema especialista educacional para validação de compatibilidade de *hardware* fundamentado em CSP. O sistema proposto é uma aplicação *web full-stack* capaz de validar a compatibilidade entre componentes de computador *desktop* e, para cada incompatibilidade detectada, comunicar ao usuário a regra técnica violada em linguagem natural. Ao transformar cada bloqueio em uma oportunidade de aprendizagem, o sistema busca não apenas restringir escolhas incorretas, mas educar o usuário sobre as restrições físicas que regem a montagem de computadores.

1.1 Justificativa

A montagem de um computador *desktop* exige do usuário o domínio de um conjunto especificações técnicas que, na prática, está além do conhecimento da maioria dos consumidores. As restrições que regem a compatibilidade entre componentes são objetivos e documentadas pelos próprios fabricantes (Advanced Micro Devices, 2022; Intel Corporation, 2023; JEDEC Solid State Technology Association, 2020), mas sua compreensão pressupõe familiaridade com conceitos cmo soquetes de processador, pradrões de memória e dimensionamento energético. Para o usuário leigo, essa barreira técnica se manifesta de forma concreta, erros de seleção, como a combinação de um processador AM5 com uma placa-mãe AM5 ou de módulo DDR4 com uma plataforma que suporta exclusivamente DDR5, resultam em incompatibilidade absoluta e em prejuízo financeiro.

As plataformas comerciais disponíveis atualmente buscam mitigar esse problema, mas como uma abordagem limitada. O PC Part Picker e o Meupc.net realizam verificações de compatibilidade e alertam o usuário quando um conflito é detectado (PCPartPicker LLC, 2024; Meupc.net, 2024). No entanto, como demonstrado por Clancey (1983), a limitação pedagógica desse modelo é nítida, um sistema que apenas filtra não transfere conhecimento, apenas restringe escolhas. Um usuário que utiliza essas plataformas repetidamente aprende quais componentes são compatíveis entre si, mas não aprende *"por que"* são compatíveis, o que o impede de generalizar o conhecimento adquirido para novas situações de compra.

Essa lacuna motivou o questionamento central deste trabalho: seria possível desenvolver um sistema que realizasse a validação de compatibilidade de *hardware* com um

motor de inferência formal, baseado no formalista de CSP, q que comunicasse ao usuário cada incompatibilidade detectada por meio de uma justificativa técnica em linguagem natural? A resposta a essa pergunta,

1.2 Objetivos

1.2.1 Objetivo Geral

1.2.2 Objetivos Específicos

- Objetivo específico 1;
- Objetivo específico 2;
- Objetivo específico n.

1.3 Problema (Hipótese ou Questão de Pesquisa)

1.4 Organização Deste Trabalho

2 REVISÃO DA LITERATURA

2.1 Compatibilidade de Hardware como Domínio do Problema

A montagem de um computador *desktop* exige o cruzamento de especificações técnicas entre componentes interdependentes. Diferentemente da aquisição de periféricos genéricos, a configuração de um sistema computacional envolve restrições físicas, elétricas e lógicas que, quando violadas, resultam em incompatibilidade absoluta: componentes adquiridos de forma incorreta não funcionam juntos, independentemente de ajustes de *software* ou configuração. Esse cenário impõe ao usuário leigo um desafio técnico significativo, uma vez que plataformas comerciais tradicionais não oferecem mecanismos de validação preventiva capazes de explicar o motivo da incompatibilidade.

O presente trabalho concentra-se em cinco categorias de componentes que constituem o núcleo de qualquer configuração *desktop*: a Unidade Central de Processamento (CPU), a placa-mãe, os módulos de memória RAM, a Unidade de Processamento Gráfico (GPU) e a fonte de alimentação. Cada uma dessas categorias possui especificações técnicas que determinam sua compatibilidade com as demais, formando uma rede de dependências que não pode ser avaliada de forma isolada.

2.1.1 Compatibilidade de Soquete CPU-Placa-mãe

A restrição mais fundamental na montagem de um computador é a compatibilidade de soquete entre o processador e a placa-mãe. Cada geração de processadores é projetada para um soquete físico específico, definido pelo número, disposição e função de seus contatos elétricos. Essa restrição é binária e absoluta: um processador projetado para o soquete AM5 da AMD é fisicamente incompatível com uma placa-mãe equipada com soquete AM4, e vice-versa, sem qualquer possibilidade de adaptação ou ajuste (Advanced Micro Devices, 2022). De forma análoga, processadores Intel para o soquete LGA1851 não são compatíveis com plataformas LGA1700 (Intel Corporation, 2023). A documentação técnica oficial dos fabricantes define formalmente quais combinações de processadores e *chipsets* são suportadas, constituindo a principal fonte de restrições binárias do domínio.

2.1.2 Padrão de Memória RAM

Os módulos de memória RAM seguem padrões técnicos definidos pela JEDEC (*Joint Electron Device Engineering Council*), organização responsável pela padronização internacional de semicondutores. Os padrões DDR4 e DDR5, por exemplo, diferem em números de pinos, tensão de operação, frequência base e formato físico do encaixe (JEDEC

Solid State Technology Association, 2020). Essa diferença torna os dois padrões eletricamente e fisicamente incompatíveis: uma placa-mãe projetada para DDR5 não aceita módulos DDR4, e o próprio conector possui geometria distinta que impede a inserção física incorreta. A seleção de memória compatível depende, portanto, da placa-mãe selecionada, que por sua vez depende do processador, criando uma cadeia de dependências transitivas no grafo de restrições.

2.1.3 Capacidade Energética e TDP

Cada componente de *hardware* possui um indicador de consumo máximo de energia denominado TDP (*Thermal Design Power*), expresso em watts. A fonte de alimentação deve ser dimensionada para suprir a soma dos TDPs de todos os componentes, com uma margem de segurança recomendada de aproximadamente 20% a 30% acima do consumo estimado (Intel Corporation, 2022). Uma fonte subdimensionada provoca instabilidade do sistema, reinicializações inesperadas e, em casos extremos, danos permanentes aos componentes. Essa restrição difere das anteriores por ser de natureza quantitativa: não se trata de uma incompatibilidade binária, mas de uma relação de desigualdade entre a capacidade da fonte e a demanda agregada dos componentes.

2.1.4 Formalização como Problema de Restrições

As três categorias de restrições descritas, soquete, padrão de memória e capacidade energética, possuem uma estrutura comum: cada um envolve um par de componentes e define condições que os valores atribuídos a esse par devem satisfazer. Essa estrutura é precisamente o que a teoria dos Problemas de Satisfação de Restrições (CSP) formaliza: cada categoria de componente pode ser modelada como uma variável, o conjunto de modelos comercialmente disponíveis em cada categoria constitui o domínio dessa variáveis. Essa correspondência torna o CSP não apenas uma metáfora conveniente, mas a estrutura formal mais adequada para modelar o problema de validação de configurações de *hardware*, conforme detalhado na seção seguinte.

2.2 Problemas de Satisfação de Restrições (CSP)

Um Problema de Satisfação de Restrições (CSP) é formalmente caracterizado por uma tripla $\langle X, D, C \rangle$, na qual $X = \{X_1, \ldots, X_n\}$ denota um conjunto finito de variáveis, $D = \{D_1, \ldots, D_n\}$ representa os respectivos domínios, de modo que cada D_i delimita os valores admissíveis para X_i, e C reúne as restrições que impõem condições sobre as combinações válidas de valores (Russell; Norvig, 2010). Cada restrição C_j é expressa como um par $\langle \mathit{scope}, \mathit{rel} \rangle$, em que *scope* é uma tupla de variáveis participantes e *rel* define os valores que essas variáveis podem assumir conjuntamente (Russell; Norvig, 2010). Diz-se

que o problema está resolvido quando se obtém uma atribuição completa e consistente: uma função que associa a cada variável um valor de seu domínio sem infringir nenhuma restrição de C.

2.2.1 Tipos de Restrições

As restrições em um CSP podem ser classificadas quanto ao número de variáveis que envolvem. Restrições unárias afetam uma única variável, limitando seu domínio independentemente das demais, por exemplo: a restrição de que a fonte de alimentação deve ter potência mínima de 500W. Restrições binárias envolvem duas variáveis e são as mais comuns em problemas de compatibilidade de *hardware*, por exemplo: a restrição de que o soquete da CPU deve corresponder ao soquete suportado pela placa-mãe. Restrições de ordem superior envolvem três ou mais variáveis simultaneamente (Dechter, 2003). O sistema proposto neste trabalho opera predominantemente com restrições binárias, o que viabiliza a representação estrutural em grafo de restrições.

2.2.2 Aplicação ao Domínio de Hardware

A aplicação dessa formalização ao domínio de compatibilidade de *hardware* é direta. Considerando os cinco componentes centrais do sistema (CPU, placa-mãe, RAM, GPU e fonte), a tripla (X, D, C) pode ser definida da seguinte forma:

- $X = \{\text{CPU}, \text{Placa-mãe}, \text{RAM}, \text{GPU}, \text{Fonte}\}$;
- $D = \{D_{\text{CPU}} = \text{modelos de processadores disponíveis},$
  $D_{\text{Placa}} = \text{modelos de placas-mãe disponíveis},$
  $D_{\text{RAM}} = \text{modelos de memória RAM disponíveis},$
  $D_{\text{GPU}} = \text{modelos de placas de vídeo disponíveis},$
  $D_{\text{Fonte}} = \text{modelos de fontes disponíveis}\}$
- $C = \{\text{soquete}(\text{CPU}, \text{Placa-mãe}),$
  $\text{padrão_memória}(\text{Placa-mãe}, \text{RAM}),$
  $\text{tdp_cpu}(\text{CPU}, \text{Fonte}),$
  $\text{tdp_gpu}(\text{GPU}, \text{Fonte})\}$

Para ilustrar concretamente, considere a atribuição parcial $\{\text{CPU} = \text{Ryzen 5 7600}, \text{Placa-mãe} = \text{B650}, \text{RAM} = \text{DDR5-5600}\}$. Essa atribuição é consistente: o Ryzen 5 7600 utiliza soquete AM5, a placa B650 suporta AM5, e o padrão DDR5 é compatível com a plataforma AM5. Contudo, a atribuição $\{\text{CPU} = \text{Ryzen 5 7600}, \text{Placa-mãe} = \text{B650}, \text{RAM} = \text{DDR4-3200}\}$ viola a restrição padrão_memória, pois placas-mãe com chipset

B650 suportam exclusivamente DDR5 (Advanced Micro Devices, 2022; JEDEC Solid State Technology Association, 2020). A solução do CSP consiste em encontrar uma atribuição completa, cobrindo todas as cinco variáveis, que satisfaça simultaneamente todas as restrições de C.

2.2.3 Representação em Grafo

A principal distinção entre o CSP e as abordagens clássicas de busca em espaço de estados reside na representação fatorada que aquele adota: em vez de tratar o estado como uma entidade atômica e indivisível, o CSP expõe a estrutura interna do problema por meio de variáveis e restrições explícitas (Russell; Norvig, 2010; Kumar, 1992). Restrições binárias entre pares de variáveis admitem representação natural como grafo, no qual os vértices correspondem às variáveis e as arestas codificam as restrições. Essa estrutura de grafo de restrições é o objeto central sobre o qual os algoritmos de busca e propagação operam.

2.3 Teoria dos Grafos e Grafo de Restrições

2.3.1 Definições Fundamentais

Um grafo é formalmente definido como um par ordenado $G = (V, E)$, onde V é um conjunto finito não vazio de vértices e $E \subseteq V \times V$ é um conjunto de arestas (Diestel, 2017). Um grafo é dito simples quando não contém laços (arestas de um vértice para si mesmo) nem arestas paralelas (múltiplas arestas entre o mesmo par de vértices). Em um grafo não direcionado, as arestas não possuem orientação: a aresta $\{u, v\}$ é idêntica a $\{v, u\}$. Dois vértices u e v são ditos adjacentes se existe uma aresta $\{u, v\} \in E$ conectando-os (Diestel, 2017).

2.3.2 Representações Computacionais

Do ponto de vista computacional, um grafo pode ser representado de duas formas principais (Cormen *et al.*, 2001). A matriz de adjacência é uma matriz A de dimensão $|V| \times |V|$, na qual $A[i][j] = 1$ se existe aresta entre os vértices i e j, e 0 caso contrário. Essa representação permite verificar a existência de uma aresta em tempo constante $O(1)$, mas exige espaço $O(|V|^2)$, independentemente do número de arestas. A lista de adjacência associa a cada vértice uma lista de seus vizinhos, exigindo espaço $O(|V| + |E|)$. Para grafos esparsos, onde o número de arestas é significativamente menor que V^2, a lista de adjacência é preferível em termos de eficiência de memória (Cormen *et al.*, 2001).

O grafo de restrições do sistema proposto é esparso por natureza: com cinco vértices (componentes), o número máximo de arestas seria 10, mas o sistema possui apenas quatro

restrições binárias efetivas, soquete entre CPU e Placa-mãe, padrão de memória entre Placa-mãe e RAM, TDP da CPU e Fonte, e TDP da GPU e Fonte. Pares como GPU e RAM ou CPU e GPU, não possuem restrições física direta e portanto não são conectados por arestas. A lista de adjacência é, portanto, a representação mais adequada para esse grafo.

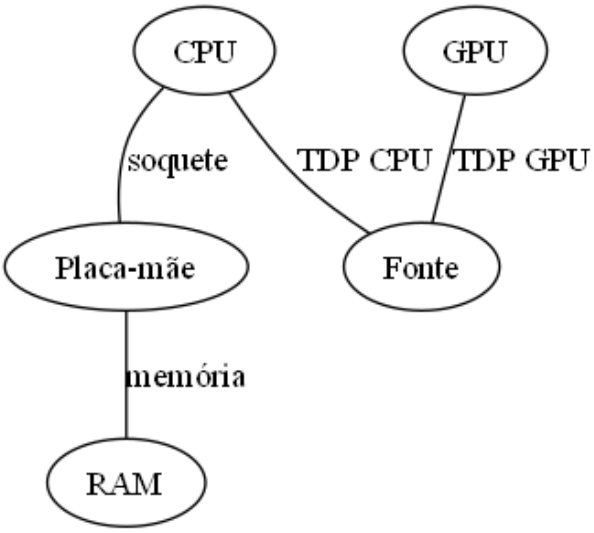
2.3.3 Grafo de Restrições

Dado um CSP com variáveis X e restrições C, o grafo de restrições $G_C = (X, E)$ é construído de modo que existe uma aresta $\{X_i, X_j\} \in E$ se e somente se existe alguma restrição C envolvendo simultaneamente as variáveis X_i e X_j (Russell; Norvig, 2010). Essa representação explicita a estrutura topológica do problema, vértices isolados correspondem a variáveis sem restrições com outras, e componentes fortemente conectados indicam grupos de variáveis com alta interdependência.

Aplicando essa definição ao sistema proposto, o grafo de restrições possui a seguinte estrutura:

- **Vértices:** $V = \{\text{CPU}, \text{Placa-mãe}, \text{RAM}, \text{GPU}, \text{Fonte}\}$
- **Arestas:** E compreende as seguintes relações:
  - $\{\text{CPU}, \text{Placa-mãe}\}$, restrição de soquete;
  - $\{\text{Placa-mãe}, \text{RAM}\}$, restrição de padrão de memória;
  - $\{\text{CPU}, \text{Fonte}\}$, restrição de TDP do processador;
  - $\{\text{GPU}, \text{Fonte}\}$, restrição de TDP da placa de vídeo.

Figura 1 – Grafo de Restrições do Sistema Proposto



Fonte: Elaborada pelo autor

Essa estrutura revela que a Fonte é um vértice de alta centralidade no grafo, pois está conectada tanto à CPU quanto à GPU, de modo que qualquer alteração no domínio de um desses componentes pode propagar restrições para o domínio da Fonte. A Placa-mãe, por sua vez, é o vértice que medeia a compatibilidade entre CPU e RAM, funcionando como ponto de articulação da rede de restrições. Esse grafo é o objeto sobre o qual os algoritmos de busca e propagação operam, cuja complexidade computacional é analisada na seção seguinte.

2.4 Complexidade Computacional

A teoria da complexidade computacional ocupa-se da classificação de problemas de decisão segundo os recursos algorítmicos, principalmente tempo de execução e espaço de memória, exigidos para sua resolução (Cormen *et al.*, 2001). Dentro desse arcabouço, a classe P reúne os problemas para os quais existem algoritmos determinísticos de tempo polinomial, considerados eficientes e tratáveis mesmo para entradas de grande escala. A classe NP, por sua vez, abrange os problemas cuas soluções propostas podem ser verificados em tempo polinomial por uma máquina determinística, ainda que não se conheça, até o presente, algoritmo eficiente para encontrá-las (Sipser, 2012). A questão sobre a relação entre essas duas classes, se $P = NP$, permanece em aberto e constitui o problema não resolvido de maior relevância para a ciência da computação teórica, razão pela qual o *Clay Mathematics Institute* o incluiu entre os sete Problemas do Milênio (Sipser, 2012).

O interior da classe NP não é uniforme. A literatura demonstra a existência de subclasses com diferentes graus de dificuldade, incluindo problemas resolúveis em tempo subexponencial por meio de não-determinismo limitado (Papadimitriou; Yannakakis,

1996). O CSP, contudo, não se enquadra nessas classes intermediárias, situando-se no topo da hierarquia como um problema NP-completo (Sipser, 2012). Um problema é classificado como NP-completo quando pertence a NP e todo problema NP se reduz a ele em tempo polinomial, essa classificação pode ser estabelecida por redução polinomial a partir do Problema da Satisfabilidade Booleana (SAT), um dos problemas canônicos de NP-completude (Cormen *et al.*, 2001; Sipser, 2012). Consequentemente, se qualquer problema NP-completo admitisse solução em tempo polinomial, toda classe NP colapsaria em P (Sipser, 2012).

2.4.1 Implicações para o Domínio de Hardware

Essa natureza NP-completa impõe restrições severas à arquitetura de qualquer sistema de verificação de compatibilidade de *hardware* fundamentado em CSP. A primeira limitação é de ordem temporal, a avaliação exaustiva de todas as combinações interdependentes de componentes é inviável na prática em razão da explosão combinatória intrínseca à classe (Sipser, 2012; Cormen *et al.*, 2001). Para ilustrar concretamente, considere que cada categoria de componente possui em média 50 modelos disponíveis no catálogo do sistema. Com cinco categorias (CPU, Placa-mãe, RAM, GPU e Fonte), a avaliação por força bruta implicaria examinar $50^5 = 312.500.000$ atribuições candidatas antes de encontrar uma solução válida no pior caso. Esse número cresce exponencialmente à medida que o catálogo se expande, tornando a abordagem inviável em tempo hábil.

A segunda limitação é de ordem espacial, estratégias alternativas baseadas no mapeamento estático e exaustivo de regras lógicas condicionais, codificando explicitamente cada restrição de *hardware* por meio de estruturas if-else ou tabela de decisão, não contornam o problema, pois a representação de funções lógicas complexas por árvores de decisão pode exiger espaço de tamanho exponencial (Jukna *et al.*, 1999). Diante dessas limitações, a abordagem padrão para tornar CSPs tratáveis na prática é a busca sistemática com retrocesso combinada com técnicas de propagação de restrições, que reduzem o espaço efetivo de busca de forma incremental.

2.5 Algoritmos de Busca e Backtracking

2.5.1 Busca em Espaço de Atribuições

Dado o grafo de restrições e a complexidade NP-completa do CSP, é necessário um algoritmo de busca sistemática que explore o espaço de atribuições de forma inteligente, sem enumerar cegamente todas as combinações possíveis. O ponto de partida é o conceito de atribuição parcial, uma função que associa valores a um subconjunto das variáveis, podendo ser progressivamente estendida até uma atribuição completa (Russell; Norvig, 2010). A ideia central dos algoritmos de busca para CSP é construir a atribuição incrementalmente,

verificando consistência a cada passo e abandonando ramos da árvore de busca assim que uma inconsistência é detectada.

2.5.2 Backtracking

O algoritmo de *backtracking* é o método fundamental de busca para CSPs (Russell; Norvig, 2010; Kumar, 1992). Ele percorre recursivamente a árvore de atribuições, em cada nível da recursão, seleciona uma variável ainda não atribuída, tenta cada valor do seu domínio na ordem disponível, verifica se esse valor é consistente com as restrições envolvendo as variáveis já atribuídas e recua (*backtrack*) ao encontrar uma inconsistência. O pseudocódigo do algoritmo é apresentado a seguir:

Algoritmo 1 – Algoritmo de Busca por Backtracking para CSP

```
1: função BACKTRACKING-SEARCH(csp)
2:     retornar BACKTRACK(∅, csp)
3: fim função

4: função BACKTRACK(atribuicao, csp)
5:     se atribuicao está completa então
6:         retornar atribuicao
7:     fim se
8:     var ← SELECIONAR-VARIAVEL-NAO-ATRIBUÍDA(csp)
9:     para cada valor em ORDENAR-VALORES(var, atribuicao, csp) faça
10:        se valor é consistente com atribuicao dado csp então
11:            adicionar {var = valor} à atribuicao
12:            resultado ← BACKTRACK(atribuicao, csp)
13:            se resultado ≠ falha então
14:                retornar resultado
15:            fim se
16:            remover {var = valor} da atribuicao
17:        fim se
18:    fim para
19:    retornar falha
20: fim função
```

Fonte: Adaptado de: Russell e Norvig (2010)

A complexidade de pior caso do *backtracking* puro é $O(d^n)$, na qual d é o tamanho máximo dos domínios e n é o número de variáveis, reiterando a inviabilidade para instâncias grandes sem otimizações complementares (Russell; Norvig, 2010).

2.5.3 Forward Checking

Uma primeira melhoria ao backtracking puro é o *forward checking* (Dechter, 2003). Ao atribuir um valor à variável X_i, o algoritmo imediatamente examina cada variável X_j, ainda não atribuída que compartilha uma restrição com X_i e remove de $Dom(X_j)$ todos os valores que se tornam inconsistentes com a atribuição de X_i. Se algum domínio X_j se torna vazio, o algoritmo detecta a falha antecipadamente e recua, sem precisar explorar o ramo até o fim.

Aplicando ao domínio de *hardware*, ao selecionar uma CPU com soquete AM5, o *forward checking* percorre os vizinhos de CPU no grafo de restrições e remove do domínio de Placa-mãe todos os modelos com soquete incompatível com AM5, antes mesmo de tentar atribuir um valor a Placa-mãe. Essa poda antecipada reduz significativamente o número de atribuições exploradas. Contudo, o *forward checking* verifica apenas os vizinhos diretos da variável recém-atribuída, sem propagar as consequências dessas remoções para variáveis mais distantes no grafo. A forma mais completa e eficiente de propagação, que garante consistência global na rede, é o Algoritmo AC-3.

2.6 Propagação de Restrições e Algoritmo AC-3

A principal técnica para contornar a explosão combinatória é a propagação de restrições, realizada por algoritmos de consistência. O mais consolidado na literatura é o AC-3 (*Arc Consistency Algorithm 3*), proposto por (Mackworth, 1977), cujo objetivo é garantir a consistência de arco em redes de restrições binárias (Kumar, 1992). Diz-se que um arco (X_i, X_j) é consistente quando, para todo valor $v \in Dom(X_i)$, existe ao menos um valor $w \in Dom(X_j)$ tal que a atribuição $X_i = v, X_j = w$ satisfaz a restrição entre as duas variáveis.

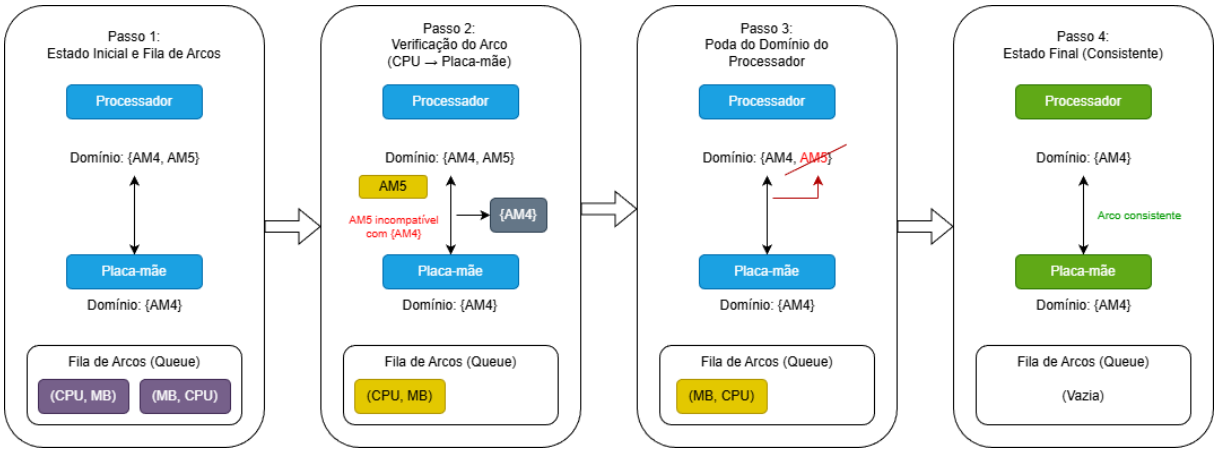
O algoritmo mantém uma fila de arcos a examinar e, para cada par de variáveis relacionadas, remove do domínio da variável os valores que não encontram suporte em nenhum valor do domínio da variável vizinha. Cada remoção dispara a reinserção dos arcos afetados na fila, propagando a mudança pela rede até o ponto de estabilidade, em que nenhuma redução adicional é possível (Russell; Norvig, 2010). Assumindo um CSP com n variáveis, domínios de tamanho máximo d e c restrições binárias, a complexidade de pior caso do AC-3 é $O(cd^3)$ (Russell; Norvig, 2010).

2.6.1 Aplicação ao Domínio de Hardware

Para ilustrar a execução do AC-3 no sistema proposto, considera-se o cenário em que o usuário seleciona um processador com soquete AM5. A Figura **??** apresenta os quatro passos da execução do algoritmo sobre o grafo de restrições formado pelas variáveis

Processador e Placa-mãe.

Figura 2 – Execução passo a passo do AC-3 no grafo de hardware



Fonte: Elaborada pelo autor

No **Passo 1**, o estado inicial é estabelecido: o domínio do Processador contém dois valores, AM4 e AM5, e o domínio da Placa-mãe contém apenas AM4. A fila de arcos é inicializada com os dois arcos direcionados da restrição bínária: (CPU, MB) e (MB, CPU).

No **Passo 2**, o arco (CPU, MB) é retirado da fila para verificação. O algoritmo examina cada valor do domínio do Processador e verifica se existe, no domínio da Placa-mãe, ao menos um valor que satisfaça a restrição de soquete. Para o valor AM4, existe suporte no domínio da Placa-mãe. Para o valor AM5, não existe suporte, pois a placa disponível suporta apenas AM4 — a incompatibilidade é identificada.

No **Passo 3**, o valor AM5 é removido do domínio do Processador por não possuir suporte na variável vizinha. O arco (MB, CPU) é reinserido na fila para que o algoritmo verifique se as remoções realizadas tornaram algum valor da Placa-mãe inconsistente.

No **Passo 4**, o arco (MB, CPU) é processado. O valor AM4 da Placa-mãe encontra suporte no valor AM4 do Processador, portanto nenhuma remoção adicional é necessária. A fila esvazia e o algoritmo encerra no ponto de estabilidade: ambas as variáveis possuem domínios consistentes com a restrição, sem necessidade de explorar nenhuma atribuição completa por força bruta.

O resultado ilustra o mecanismo central do motor de inferência: uma única seleção do usuário dispara a propagação de restrições por toda a rede, reduzindo os domínios disponíveis de forma incremental e em tempo real (Mackworth, 1977; Russell; Norvig, 2010).

2.7 Sistema Especialista

Um Sistema Especialista (SE) é um programa de computador projetado para simular o raciocínio de um especialista humano em um domínio restrito e bem definido (Russell; Norvig, 2010). Diferentemente de sistema de propósito geral, um SE concentra seu conhecimento em um domínio específico e é capaz de resolver problemas nesse domínio com nível de competência comparável ao de um especialista humano (Jackson, 1998). Sua arquitetura é composta por três elementos centrais: a base de conhecimento, que armazena fatos e regras sobre o domínio, o motor de inferência, que aplica essas regras aos fatos disponíveis para derivar conclusões e a interface de explicação, que permite ao sistema justificar suas respostas ao usuário (Russell; Norvig, 2010).

O motor de inferência pode operar segundo encadeamento progressivo (*forward chaining*), partindo dos fatos conhecidos em direção a conclusões, ou encadeamento regressivo (*backward chaining*), partindo de uma hipótese e verificando se os fatos disponíveis a sustentam (Russell; Norvig, 2010). No sistema proposto, o motor opera predominantemente em modo progressivo: a cada seleção de componente pelo usuário, o sistema propaga as restrições a partir desse novo fato para derivar quais opções permanecem válidas.

2.7.1 Explanation Facilities

A capacidade de justificar o próprio raciocínio, denominada na literatura como *explanation facility*, é o que distingui fundamentalmente um SE de um mecanismo de filtragem comum (Clancey, 1983). Segundo (Clancey, 1983), essa capacidade exige que o sistema mantenha não apenas as conclusões derivadas, mas também a cadeia de regras e restrições que levou a cada conclusão, de modo que possa reconstruir e comunicar o raciocínio em linguagem acessível ao usuário. Um mecanismo de filtragem simples responde apenas à pergunta "quais opções são válidas?", apresentado resultados sem elaboração. Um SE com *explanation facility* responde também à pergunta "por que as demais opçõs são inválidas?", articulando a regra específica que foi violada.

No contexto do sistema proposta, essa capacidade se manifesta da seguinte forma, ao bloquear a seleção de um módulo DDR4 após a escolha de uma CPU AM5, o sistema não se limita a remover o componente da lista de opções, ele apresenta ao usuário uma justificativa explícita como "Este módulo de memória é incompatível: a placa-mãe compatível com o processador selecionada suporta exclusivamente DDR5 e este módulo é DDR4". Essa explicação transforma cada bloqueio em uma oportunidade de aprendizado sobre as restrições físicas do *hardware*.

2.7.2 Inteligência Artificial Explicável (XAI)

O problema identificado por Clancey (1983), a necessidade de sistemas inteligentes justificarem seu raciocónio ao usuário, permaneceu latente por décadas e ressurgiu com força renovada no contexto da Inteligência Artificial (IA) moderna sob o nome de XAI (*eXplainable Artificial Intelligence*), ou Inteligência Artificial Explicável. Barredo Arrieta *et al.* (2020) definem XAI como o conjunto de processos e técnicas que permitem que modelos de inteligência artificial produzam resultados compreensíveis por seres humanos, especialmente por usuários finais sem formação técnica em IA. Os autores identificam a explicabilidade como uma necessidade crítica para a adoção prática de sistemas de IA em domínios sensíveis, argumentando que um sistema que apenas produz resultados sem justificá-los não gera confiança suficiente para ser adotado em decisões reais.

A distinção central que o campo de XAI estabelece é entre sistemas caixa-preta (*black-box*), cujo processo de decisão interno é opaco ao usuário, e sistemas transparentes, cujo raciocínio pode ser inspecionado e compreendido (Barredo Arrieta *et al.*, 2020). Sistemas especialistas baseados em CSP pertencem naturalmente à segunda categoria, as variáveis, os domínios e as restrições são explícitos e auditáveis e cada decisão de bloqueio pode ser rastreada até a restrição específica que a originou. Nesse sentido, o motor de inferência CSP/AC-3 proposto neste trabalho é intrinsecamente explicável por *design*, não por técnica de pós-processamento, mas pela própria natureza simbólica do modelo. Isso posiciona o sistema proposto em alinhamento com os princípios contemporâneos de XAI, cada resultado produzido pelo motor de inferência é acompanhado da justificativa técnica que o fundamenta, tornando o processo de validação de *hardware* transparente e auditável pelo usuário.

2.7.3 Mapeamento ao Sistema Proposto

O sistema proposto neste trabalho se enquadra na categoria de sistemas especialistas baseados em restrições. Os três componentes arquiteturais do SE mapeiam diretamente à implementação: a base de conhecimento corresponde à camada de dados que armazena as especificações técnicas dos componentes e as regras de compatibilidade modeladas como restrições do CSP; o motor de inferência corresponde ao algoritmo AC-3 implementado na camada de lógica da aplicação, responsável por propagar restrições e podar domínios a cada interação do usuário; e a interface de explicação corresponde aos alertas técnicos e educativos exibidos na camada de apresentação, que comunicam ao usuário não apenas quais componentes são bloqueados, mas as razões físicas e técnicas subjacentes a cada bloqueio (Jackson, 1998).

2.8 Dimensão Educativa em Sistemas Especialistas

O caráter educativo do sistema proposto não decorre da adoção de uma arquitetura de aprendizagem adaptativa, mas da forma como o motor de inferência comunica suas conclusões ao usuário. Para delimitar com precisão essa dimensão, é necessário distinguir três categorias de sistemas que operam sobre domínios de conhecimento restrito: os mecanismos de filtragem, os Sistemas Tutores Inteligentes (STI) e os Sistemas Especialistas com interface educativa.

2.8.1 Mecanismo de Filtragem versus Sistema com Explicação

Um mecanismo de filtragem tradicional opera sobre um conjunto de dados e retorna os elementos que satisfazem determinados critérios, sem qualquer elaboração sobre o processo de decisão. No contexto de compatibilidade de *hardware*, um filtro responde à pergunta "quais componentes são compatíveis com minha seleção atual?", apresentando uma lista de resultados válidos e ocultando ou desabilitando os inválidos, sem indicar o motivo da exclusão. Plataformas comerciais como o *PC Part Picker* operam predominantemente nesse paradigma: informam a incompatibilidade, mas raramente explicam a regra física que a origina.

A limitação pedagógica desse modelo é nítida: o usuário aprende o que não pode fazer, mas não aprende o motivo. Cada bloqueio silencioso é uma oportunidade de aprendizagem perdida (Clancey, 1983). Um sistema que apenas filtra não transfere conhecimento, apenas restringe escolhas.

2.8.2 Sistemas Tutores Inteligentes

Sistemas Tutores Inteligentes (STI) representam o paradigma mais sofisticado de instrução mediada por computador. Sua arquitetura formal é composta por quatro modelos interdependentes: o modelo de domínio, que representa o conhecimento a ser ensinado; o modelo de estudante, que representa o estado de conhecimento individual do aprendiz e é atualizado continuamente com base em seus erros e acertos; o modelo pedagógico, que define as estratégias de instrução e adaptação; e o modelo da interface, que define como o conteúdo é apresentado (Woolf, 2008). O elemento central que distingue um STI de outros sistemas é o modelo de estudante, ele exige que o sistema mantenha um perfil persistente por usuário, rastreando o histórico de interações ao longo de múltiplas sessões para personalizar a instrução (Anderson *et al.*, 1995).

O sistema proposto neste trabalho não implementa a arquitetura de um STI. Não há rastreamento de sessões individuais, não há perfil persistente de usuário e não há adaptação sequencial de conteúdo baseado em histórico. Essa delimitação é intencional, pois o objetivo é criar um sistema especialista com interface educativa, que se concentra na

explicação de cada bloqueio como uma oportunidade de aprendizagem, sem a complexidade adicional de um modelo de estudante adaptativo.

2.8.3 Sistema Especialista Educacional

O sistema proposto se enquadra na categoria de Sistema Especialista Educacional, um SE dotado de *explanation facility* cuja interface é projetado com objetivo pedagógico explícito. A distinção em relação a um SE convencional reside na forma como as conclusões são comunicadas. Enquanto um SE padrão pode simplesmente retornar um resultado binário (compatível ou incompatível), um SE educacional articula a cadeia de raciocínio que levou aquela conclusão em linguagem acessível ao usuário (Clancey, 1983; Jackson, 1998).

No sistema proposto, essa dimensão educativa se manifesta de forma precisa e delimitada: ao detectar uma violação de restrição via propagação AC-3, o sistema não limita a bloquear o componente incompatível, ele apresenta ao usuário uma justificativa explícita que identifica qual restrição foi violada e por qual razão técnica. Por exemplo, ao tentar selecionar um módulo DDR4 após escolher umas CPU com plataforma AM5, o sistema exibe: "Este módulo de memória é incompatível: a placa-mãe compatível com o processador selecionado suporta exclusivamente DDR5, e este módulo utiliza o padrão DDR4". Essa explicação transforma cada bloqueio em uma oportunidade de aprendizagem sobre os princípios físicos que regem a montagem de computadores, sem requerer qualquer cadastro, histórico ou personalização por parte do usuário.

Essa abordagem é pedagogicamente coerente com público-alvo do sistema, o usuário leigo que realiza uma única sessão de configuração, para quem a explicação contextualizada e imediata é mais eficaz do que qualquer mecanismo de adaptação baseado em histórico. O adjetivo "Educacional"no título do trabalho refere-se precisamente a essa capacidade de instruir o usuário sobre as restrições do domínio no exato momento em que elas se tornam relevantes para sua decisão.

2.9 Arquitetura de Software Orientada a Serviços

2.9.1 Arquitetura em Camadas

A arquitetura de *software* compreende o conjunto de decisões estruturais fundamentais que definem a organização de um sistema, as responsabilidades de seus componentes e as interfaces através das quais eles se comunicam (Fowler, 2002). Um dos padrões arquiteturais mais consolidados na engenharia de *software* é a arquitetura em camadas, que organiza o sistema em níveis hierárquicos de abstração, cada um com responsabilidades

bem delimitadas e comunicando-se apenas com a camada imediatamente adjacente (Fowler, 2002).

As três camadas clássicas são: a camada de apresentação, responsável pela interação com o usuário e pela exibição de informações, a camada de lógica de negócio, responsável pelo processamento, pelas regras do domínio e pelas decisões da aplicação e a camada de dados, responsável pelo armazenamento, recuperação e persistência de informações (Fowler, 2002). Essa separação aumenta a manutenibilidade do sistema, alterações em uma camada não propagam mudanças para as demais, e facilita a testabilidade, permitindo que cada camada seja verificada de forma independente.

2.9.2 REST como Estilo Arquitetural

A comunicação entre camadas distribuídas em sistemas *web* segue estilos arquiteturais que impõem restrições sobre como os componentes interagem. O estilo mais amplamente adotado é o REST (*Representational State Transfer*), definido por (Fielding, 2000) em sua tese de doutorado como um conjunto de seis princípios arquiteturais para sistemas distribuídos baseados em rede.

Os princípios mais relevantes para o sistema proposto são:

O princípio de cliente-servidor, que estabelece a separação entre a interface do usuário e o armazenamento e processamento de dados, permitindo que cada lado evolua de forma independente. No sistema proposto, o cliente e o servidor evoluem independentemente, a interface pode ser redesenhada sem alterar o motor CSP, e o motor CSP pode ser otimizado sem alterar a inteface.

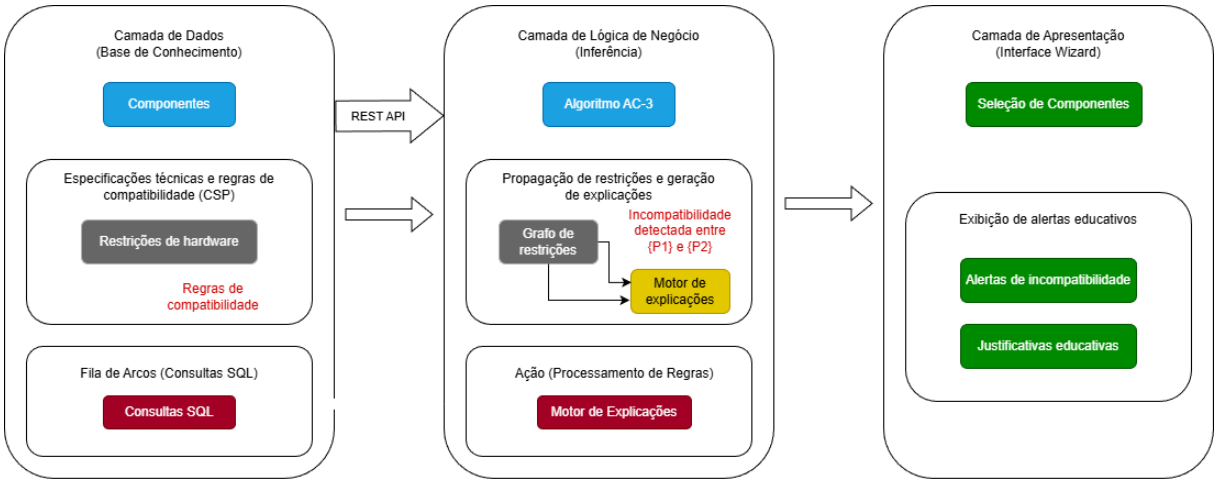
O princípio *stateless* (sem estado), que estabelece que cada requisição enviado pelo cliente ao servidor deve conter todas as informações necessárias para ser compreendida e processada, sem depender de contexto ou sessão armazenada no servidor entre requisições (Fielding, 2000). No contexto do sistema proposto, isso significa que cada chamada à *API* carrega o estado atual completo das seleções do usuário, quais componentes foram escolhidos e quais permanecem em aberto e o motor CSP executa a propagação de restrições a partir desse estado, sem manter sessão persistente no servidor. Essa característica garante escalabilidade e simplicidade operacional.

O princípio da interface uniforme, que estabelece que a comunicação entre o cliente e servidor deve seguir um contrato padronizado, baseado em recursos identificados por URIs e operações semânticas bem definidas (Richardson; Ruby, 2007). No sistema proposto, cada categoria de componente e cada conjunto de restrições é exposto como um recurso REST, acessível por operações HTTP padronizadas.

2.9.3 Mapeamento ao Sistema Proposto

A arquitetura do sistema proposto realiza concretamente os princípios da arquitetura em camadas e do estilo REST, conforme ilustrado na Figura **??**.

Figura 3 – Diagrama da arquitetura em três camadas do sistema



Fonte: Elaborada pelo autor

A **camada de dados**, representada no painel esquerdo da figura, armazena a base de conhecimento do sistema especialista: os modelos de componentes de *hardware*, as especificações técnicas que constituem as variáveis do CSP, as restrições de compatibilidade que compõem o conjunto C, e as regras de compatibilidade que fundamentam as decisões do motor de inferência. O acesso a essa camada é realizado por meio de consultas SQL disparadas pela camada de lógica de negócio.

A **camada de lógica de negócio**, representada no painel central, é o núcleo inteligente do sistema. Após receber uma requisição via *REST API* contendo o estado atual das seleções do usuário, essa camada executa o Algoritmo AC-3 sobre o grafo de restrições: identifica os arcos inconsistentes, poda os domínios das variáveis afetadas e, ao detectar uma incompatibilidade entre dois componentes P_1 e P_2, aciona o motor de explicações, que formula a justificativa técnica a ser apresentada ao usuário. O resultado do processamento é retornado à camada de apresentação como resposta à requisição REST.

A **camada de apresentação**, representada no painel direito, é a interface *wizard* com a qual o usuário interage diretamente. Ela é responsável por exibir o catálogo de componentes disponíveis para seleção, apresentar os alertas de incompatibilidade gerados pelo motor de inferência e comunicar as justificativas educativas que fundamentam cada bloqueio, transformando o resultado da validação CSP em uma oportunidade de aprendizagem sobre os princípios físicos que regem a montagem de computadores.

A separação entre as três camadas garante que o motor de inferência possa ser mantido e expandido de forma independente da interface de apresentação, preservando a

modularidade arquitetural ao longo do ciclo de desenvolvimento do sistema (Fowler, 2002; Fielding, 2000).

2.10 Trabalhos Relacionados

Esta seção mapeia o estado da arte em duas frentes: as plataformas comerciais de verificação de compatibilidade de *hardware* atualmente disponíveis e os trabalhos acadêmicos sobre sistemas especialistas e CSP aplicados a domínios similares. O objetivo é evidenciar as lacunas existentes nas soluções atuais e posicionar a contribuição do presente trabalho em relação a elas.

2.10.1 Plataformas Comerciais de Compatibilidade de Hardware

O PC Part Picker é a plataforma de referência global para montagem de computadores. Permite ao usuário selecionar componentes de um catálogo extenso e verifica automaticamente incompatibilidades entre as pelas escolhidas (PCPartPicker LLC, 2024). Quando uma incompatibilidade é detectada, a plataforma exibe um alerta genérico indicando que os componentes são incompatíveis, mas sem detalhar a regra técnica violada, por exemplo, ao selecionar uma CPU AM5 com um módulo DDR4, o sistema indica o conflito sem explicar a diferença entre os padrões de memória ou a razão física da incompatibilidade. Do ponto de vista da teoria de sistemas especialistas, o PC Part Picker implementa um mecanismo de filtragem sem *explanation facility*, o usuário recebe o resultado da validação, mas não o raciocínio que o produziu (Clancey, 1983).

O Meupc.net é a alternativa nacional mais utilizada no contexto brasileiro, operando de forma similar ao PC Part Picker com catálogo voltado ao mercado local (Meupc.net, 2024). Apresenta as mesmas limitações em termos de explicação, as incompatibilidades são sinalizadas visualmente, mas sem justificativa técnica que permita ao usuário compreender o princípio de *hardware* subjacente à restrição.

A limitação comum a ambas as plataformas é estrutural, por operaram como mecanismos de filtragem e não como sistemas especialistas com capacidade explicativa, não transferem conhecimento ao usuário, apenas restringem escolhas. Um usuário que utiliza PC Part Picker repetidamente aprende quais componentes são compatíveis entre si, mas não aprende por que são compatíveis, o que o impede de generalizar o conhecimento adquirido para novas situações de compra.

2.10.2 Trabalhos Acadêmicos Relacionados

Na literatura acadêmica, a aplicação de CSPs a problemas de configuração de produtos é um tema consolidado. Sabin e Weigel (1998) apresentam uma *survey* abrangente

sobre sistemas de configuração baseados em CSP, identificando a modelagem de dependências entre componentes como variáveis e restrições é a abordagem mais expressiva para domínios com regras de compatibilidade complexas. Os autores demonstram que sistemas de configuração baseada em CSP superam abordagem baseadas em regras estáticas tanto em expressividade quanto em eficiência de manutenção de base de conhecimento, resultado diretamente aplicável ao domínio de *hardware* proposto neste trabalho.

Mittal e Frayman (1989) introduzem o conceito de *dynamic CSP*, no qual o conjunto de variáveis e restrições pode ser alterado durante a busca, uma extensão relevante para sistemas de configuração interativos onde o usuário faz escolhas incrementais, como é o caso do sistema proposto. Essa abordagem sustenta a decisão arquitetural de propagar restrições a cada interação do usuário, em vez de processar o CSP completo apenas ao final da configuração.

No campo de sistemas especialistas educacionais, Nwana (1990) discute os requisitos para que um sistema especialista seja considerado efetivamente educativo, identificando a capacidade de explicação como condição necessária, mas não suficiente, para a transferência de conhecimento. O autor distingue sistemas que explicam o que de sistemas *o que* explicam *por que*, argumentando que apenas os últimos têm impacto pedagógico real. Esse referencial sustenta diretamente a decisão de projeto de exibir não apenas o resultado da validação, mas a regra de compatibilidade violada em linguagem acessível ao usuário.

Mais recentemente, Wang e Wu (2018) desenvolveram um sistema especialista educacional para apoio ao aprendizado de classificação botânica em ambiente móvel, demonstrando que sistemas especialistas com *feedback* explicativo produzem ganhos mensuráveis de aprendizagem em comparação com sistemas que apenas apresentam informação estática. O estudo é relevante para o presente trabalho por demonstrar empiricamente que a capacidade de explicação, e não apenas a apresentação de conteúdo, é o fator determinante para o impacto pedagógico de um sistema especialista, validando a abordagem adotada no sistema de compatibilidade de *hardware* proposto.

2.10.3 Posicionamento do Trabalho Proposto

A análise das plataformas comerciais e dos trabalhos acadêmicos permite identificar com precisão a contribuição do presente trabalho. Em relação as plataformas comerciais, o diferencial está na *explanation facility.* enquanto PC Part Picker e Meupc.net sinalizam incompatibilidades sem explicá-las, o sistema proposto articula a regra técnica violada em cada bloqueio, transformando a interação em uma oportunidade de aprendizagem. Em relação aos trabalhos acadêmicos, a contribuição está na combinação de três elementos que, individualmente, já são estudados na literatura, CSP como motor de inferência (Sabin; Weigel, 1998), sistema especialista com capacidade explicativa alinhada a princípios de XAI (Clancey, 1983; Barredo Arrieta *et al.*, 2020) e interface educativa com impacto

pedagógico demonstrável (Wang; Wu, 2018), em um sistema *web* completo validado sobre um domínio concreto e de relevância prática imediata para o usuário.

3 MATERIAIS E MÉTODOS

De acordo com a natureza e os objetivos do presente trabalho, foram adotadas as seguintes metodologias de pesquisa para embasar o desenvolvimento do sistema especialista educacional proposto.

3.0.1 Bibliográfica

Durante o desenvolvimento deste trabalho, aplica-se a metodologia bibliográfica, por meio da qual foi realizada a análise de materiais já publicados. Para isso, fez-se necessária a avaliação e fundamentação sobre os conteúdos consultados podendo eles ser "de livros, periódicos, documentos, textos, mapas, fotos, manuscritos e, até mesmo, de material disponibilizado na internet etc"(Fontelles *et al.*, 2009).

Em primeira instância, a pesquisa foi conduzida com a definição dos temas e objetivos necessários, com o intuido de identificar os assuntos que sustentam a narrativa do trabalho, esclarecendo quais pontos são úteis e relevantes para a pesquisa. Os temas centrais investigados foram: Problemas de Satisfação de Restrições (CSP), algoritmos de propagração de restrições, sistemas especialistas, inteligência artificial explicável (XAI), sistemas tutores inteligentes e arquitetura de software orientada a serviços. Além disso, foi necessário delimitar um período para a seleção das fontes, priorizando materiais mais recentes, sem desconsiderar obras clássicas e fundamentais para o domínio, como os trabalhos de Clancey (1983), Mackworth (1977) e Russell e Norvig (2010), dada a relevância teórica permanente de tais referências.

Após essa etapa inicial, foram selecionadas como principais bases de busca o Google Scholar e o Portal de Periódicos da CAPES, acessado por meio do sistema CAFe (Comunidade Acadêmica Federada), em razão de sua vasta cobertura de artigos científicos, livros, teses, dissertações e demais produções acadêmicas indexadas. Foram também realizadas buscas direcionadas em portais de conferências e periódicos especializados em inteligência artificial, engenharia de *software* e sistemas de informação.

Dado o volume de resultados retornados, realizou-se uma triagem dos materiais recuperados por meio da leitura de resumos. Os materiais considerados relevantes foram incluídos, nos casos de dúvidas quando à permanência, procedeu-se a uma leitura mais minuciosa. Após a seleção, foi realizada uma análise detalhada dos materiais e a síntese das informações coletadas.

Por fim, todas as pesquisas e extrações de informações foram revisadas, de modo a evitar erros e má interpretação. A revisão foi conduzida em duas etapas, com releitura do

material, da síntese e da extração de informação, resultando em ajustes quando necessário e garantindo a qualidade e a precisão das informações utilizadas ao longo do trabalho.

3.0.2 Aplicada

A pesquisa aplicada orienta o desenvolvimento prático de compatibilidade, cujo produto é um sistema especialista educacional para validação de compatibilidade de *hardware*. Segundo Gerhardt e Silveira (2009), a pesquisa aplicada tem como objetivo gerar conhecimento para aplicação prática e dirigidos à solução de problemas específicos. Neste contexto, os fundamentos teóricos levantados pela pesquisa bibliográfica, notadamente o formalismo de CSP, o algoritmo AC-3 e os princípios de sistemas especialistas com *explanation facility*, foram diretamente empregados na concepção e implementação do sistema proposto.

Essa abordagem permitiu que as decisões arquiteturais e de projeto fossem fundamentadas em evidências da literatura, como a adoção da arquitetura em três camadas (Fowler, 2002), o estilo REST para comunicação entre camadas (Fielding, 2000) e o algoritmo AC-3 como motor de inferência (Mackworth, 1977), garantindo o alinhamento entre os resultados práticos obtidos e o referencial teórico consolidade.

3.1 Materiais

3.1.1 Equipamentos

Para a realização do desenvolvimento, codificação e testes do sistema, foi utilizado um computador com processaor Ryzen 7 5700X, 32GB de memória RAM, placa de vídeo AMD Radeon RX 7600 e sistema operacional Windows 11. Esses recursos foram suficientes para executar o ambiente de desenvolvimento, o servidor de aplicação, o sistema de gerenciador de banco de dados e os navegadores necessários para a validação da interface.

3.1.2 Softwares

Para o desenvolvimento do sistema foram utilizadas ferramentas de *software* selecionadas com base em sua adequaçãoi técnica ao projeto, disponibilidade gratuita ou de código aberto e familiaridade prévida do desenvolvedor.

3.2 Ferramentas Utilizadas

3.2.1 Visual Studio Code

O Visual Studio Code (VS Code) é um editor de código-fonte desenvolvido pela Microsoft, disponível gratuitamente para sistemas operacionais Windows, Linux e MacOS. Por meio de extensões, a ferramenta agrega funcionalidades como depuração, controle de versão, terminal integrado e suporte a diversas linguagens de programação, aproximando-se de um ambiente de desenvolvimento integrado (IDE).

No presente trabalho, o VS Code foi utilizado como ambiente principal de desenvolvimento, tanto para codificação do *backend* e do *frontend* quanto para a edição e compilação do documento LaTeX deste trabalho, por meio da extensão LaTeX Workshop. A escolha pela ferramento deve-se à sua leveza, à vesta disponibilidade de extensões relevantes para o ecossistema adotado e ao conhecimento prévio do desenvolvedor.

3.2.2 Node.js e NestJS

O Node.js é um ambiente de execução JavaScript de código aberto e multiplataforma, construído sobre o motor V8 do Google Chrome, que permite a execução de código JavaScript no lado do servidor, sendo amplamante empregado no desenvolvimento de aplicações *web* escaláveis e de alta performance. O NestJS é um *framework* para desenvolvimento de aplicações server-side organizada em módulos, *controller* e *services*. Essa estrutura favorece a separação de responsabilidades e a manutenibilidade do código, alinhando-se diretamente aos princípios da arquitetura em camadas descrito por Fowler (2002).

No contexto do sistema proposto, o Node.js com NestJS compõe a camada de lógica de negócio, sendo o responsável por receber as requisições da interface, executar o algoritmo AC-3 sobre o grafo de restrições de *hardware* e retornar ao cliente os resultados da validação acompanhados das justificativas educativas correspondentes.

3.2.3 PostgreSQL e Docker

O PostgreSQL é um banco de dados relacional de código aberto, com suporte a transações, multiplataforma e amplamente utilizado em aplicações *web* de diferentes escalas. A escolha por esse sistema deve-se à sua robustez, à sua conformidade com o padrão SQL e à facilidade de integração com ecossistemas Node.js.

No sistema proposto, o PostgreSQL é utilizado para armazenar a base de conhecimento do sistema especialista: os modelos de componentes de *hardware*, sua especificações técnicas e as regras de compatibilidade que compõe o conjunto de restrições do CSP.

Essas informações são consultadas pela camada de lógica de negócio a cada execução do algoritmo AC-3.

O provisionamento do banco de dados foi realizado por meio do Docker, uma plataforma de código aberto para criação e execução de contêineres de *software*. A utilização do Docker permite que o ambiente do banco de dados seja definido de forma declarativa e reproduzível, garantindo consistência entre os ambientes de desenvolvimento e produção sem a necessidade de instalação direta do PostgreSQL no sistema operacional.

3.2.4 DBeaver

O DBeaver é uma ferramenta de administração de banco de dados de código aberto e multiplataforma, compatível com diversos sistemas gerenciadores, incluindo o PostgreSQL. A ferramente oferece uma interface visual para a execução de consultas SQL, navegação entre tabelas, modelagem de dados e exportação de resultados.

No presentr trabalho, o DBeaver foi utilizado para o auxílio na administração e manipulação do banco de dados durante o desenvolvimento, permitindo a inspeção e validação dos dados cadastrados na base de conhecimento do sistema especialista.

3.2.5 Next.js

O Next.js é um *framework* de código aberto para o desenvolvimento de aplicações *web* contruído sobre o React, desenvolvido e mantido pela Vercel. O *framework* oferece recursos como renderização híbrida, roteamente baseado em sistemas de arquivos e suporte nativo a APIs, facilitando o desenvolvimento de aplicações *web* com uma única base de código.

No presente trabalho, o Next.js foi utilizado para o desenvolvimento da camada de apresentação do sistema, implementada como um interface no formato *wizard* que guia o usuário, passo a passo, na seleção dos componentes de *hardware*. A escolha do Next.js deve-se à sua capacidade e integrar a interface e as chamadas à camada de lógica de negócio em um único projeto, além de atualizar dinamicamente os componentes exibidos na tela em resposta ao resultado das validações retornadas pelo motor de inferência.

3.2.6 Figma

O Figma é uma ferramenta de *design* de interface baseada em nuvem, acessada por meio de navegadore *web*, que permite a criação de protótipos e *layouts* de forma colaborativa em tempo real, durante o processo de criação é possível fazer uso de diferente modelos e ter uma visibilidade geral do projeto.

No presente trabalho, o Figma foi utilizado para o desenvolvimento dos protótipos das interfaces do sistema, permitindo planejar e validar o *layout* da interface *wizard* antes da sua implementação na camada de apresentação. A escolha pela ferramente deve-se à sua disponibilidade gratuita para uso acadêmico e ao conhecimento prévio do desenvolvedor.

3.2.7 Astah Community

O Astah Community é uma ferramenta de modelam UML desenvolvida pela empresa japonesa Change Vision, que possibilida a criação de diagramas de casos de uso, de classes, de sequência, de domínio e de atividades, entre outros. A ferramento disponibiliza uma versão gratuita voltada ao uso acadêmico.

No presente trabalho, o Astah Community foi utilizado para a elaboração dos diagramas UML, que compõe as etapas de análise e projeto de *software*, incluindo o diagrama de casos de uso, de domínio, de classes e de sequência (por enquanto).

3.2.8 Teste?

3.3 Métodos de Desenvolvimento

3.3.1 Modelagem do Problema como CSP

A etapa central do método de desenvolvimento consistiu na formalização do problema de compatibilidade de *hardware* como um Problema de Satisfação de Restrições (CSP). Conforme descrito por Russell e Norvig (2010), um CSP é caracterizado por uma tripla (X, D, C), na qual X representa o conjunto de variáveis, D os respectivos domínios e C o conjunto de restrições.

No sistema proposto, cada categoria de componente de *hardware* foi mapeada a uma variável do CSP, cujo domínio é composto pelos modelos comercialmente disponíveis cadastrados na base de conhecimento. As restrições foram identificadas a partir das especificações técinas oficientes dos fabricantes e formalizadas como restrições binárias entre pares de variáveis.

3.3.2 Implementação do Algoritmo AC-3

O motor de inferência do sistema foi implementado com base no algoritmo AC-3 (*Arc Consistency Algorithm 3*), proposto por Mackworth (1977). A implementação seguiu a estrutura algorítmica descrita por Russell e Norvig (2010), uma fila de arcos é inicializada com todos os pares de variáveis relacionadas por restrições binárias, para cada arco retirado da fila, o algoritmo verifica se todos os valores do domínio da variável de origem possuem

suporte no domínio da variável de destino, valores sem suporte são removidos do domínio e, a cada remoção, os arcos afetados são reinseridos na fila para propagação.

No contexto do sistema proposto, o AC-3 é invocado a cada seleção de componente realizada pelo usuário na interface. O estado atual das seleções é enviado à camada de lógica de negócio, implmentada com NestJS, que executa a propagação de restrições e retorna ao cliente os domínios atualizados, juntamente com as justificativas técnicas referentes a cada valor removido. Essa abordagem garante que a validação ocorra de forma incremental e em tempo real, tornando viável a execução sobre catálogos de tamanho realista sem ocorrer explosão combinatória, o que aconteceria no caso de validação por força bruta.

3.3.3 Implementação das Justificativas Educativas

Para cada restrição violada pelo AC-3, o sistema aciona um módulo de geração de justificativas, responsável por formular, em linguagem natural, a explicação técnica correspondente à incompatibilidade identificada. Esse módulo implementa o conceito de *explanation facility* descrito por Clancey (1983), que distingue um sistema especialista de um mecanismo de filtragem simples pela sua capacidade de articular a cadeia de regras e restrições que levou a uma determinada conclusão.

As justificativas foram estruturadas de forma a identificar explicitamente qual restrição foi violada, quais componentes envolvidos e qual o princípio técnico subjacente à incompatibilidade. Por exemplo, ao detectar a seleção simultânea de um processador com plataforma AM5 e um módulo de memória DDR4, o sistema exibe ao usuário a seguinte justificativa: *"Este módulo de memória é incompatível: a placa-mãe compatível com o processador selecionado suporta exclusivamente DDR5, e este módulo utiliza o padrão DDR4"*. Dessa forma, cada bloqueio é transformado em uma oportunidade de aprendizagem sobre as restrições físicas que regem a montagem de computadores.

3.3.4 Processo de Desenvolvimento de Software

O desenvolvimento do sistema seguiu uma abordagem iterativa e incremental, na qual as funcionalidades foram implmentadas e validadas em etapas sucessivas. Em cada iteração, foram realizadas as atividades de análise de requisitos, projeto, implementação e teste, permitindo a identificação e correção precoce de inconsistências entre os requisitos especificados e o comportamento do sistema.

A organização das tarefas foi realizada por meio de uma ferramente de gerenciamento de projetos, com atividades distribuídas conforme seu estado de desenvolvimento: a fazer, em desenvolvimento, em teste, concluído e adiado. Essa estrutura permitiu o acompanhamento contínuo do progresso do projeto e a priorização das atividades ao longo

do ciclo de desenvolvimento.

3.3.5 Estratégia de Testes

4 DESCRIÇÃO GERAL DO PRODUTO

5 ELICITAÇÃO DE REQUISITOS E ANÁLISE

5.1 Requisitos do Usuário

Texto da justificativa

5.2 Requisitos do Sistema

Fazer uma pequena definição de requisitos do sistema para introduzir a seção (informar a fonte). Assim, esta seção apresenta os requisitos identificados nesse trabalho.

5.2.1 Requisitos Funcionais

De acordo com **braga2008algoritmos**, Requisitos Funcionais são ...

Os atores que interagem com o sistema são: usuário e sistema. Os requisitos funcionais identificados para este sistema estão apresentados nos Quadros 1 a 18.

Os requisitos estão classificados quanto à prioridade (*Essencial*, *Importante* ou *Desejável*) e quanto à operação (*Entrada*, *Processamento* ou *Saída*).

Quadro 1 – RF-01 – Listar componentes por categoria

Código	RF-01
Grupo	Catálogo de Componentes
Descrição	**Ação:** Listar. **Objeto:** Componentes de hardware. **Atributos:** categoria (CPU, Placa-mãe, RAM, GPU, Fonte), nome do modelo, especificações técnicas (soquete, padrão DDR, TDP, potência). **Exemplos:** Usuário acessa o sistema e visualiza a lista de CPUs disponíveis; usuário avança para a etapa seguinte e visualiza a lista de placas-mãe. **Restrições:** 1. O catálogo deve ser exibido separado por categoria. 2. As categorias obrigatórias são: CPU, Placa-mãe, RAM, GPU e Fonte. 3. Cada componente deve exibir suas especificações técnicas relevantes à compatibilidade.
Prioridade	Essencial
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 2 – RF-02 – Navegar pela interface *wizard*

Código	RF-02
Grupo	Catálogo de Componentes
Descrição	**Ação:** Navegar. **Objeto:** Interface *wizard* de configuração passo a passo. **Atributos:** etapa atual, categoria de componente em seleção. **Exemplos:** Usuário conclui a seleção da CPU e avança para a etapa de Placa-mãe; usuário retorna à etapa anterior para revisar a seleção. **Restrições:** 1. A interface deve guiar o usuário sequencialmente por cada categoria de hardware. 2. O usuário deve poder avançar e retroceder entre etapas.
Prioridade	Essencial
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 3 – RF-03 – Exibir componentes incompatíveis com sinalização visual

Código	RF-03
Grupo	Catálogo de Componentes
Descrição	**Ação:** Exibir com sinalização visual de bloqueio. **Objeto:** Componentes incompatíveis com a seleção atual. **Atributos:** status de compatibilidade, indicador visual de bloqueio. **Exemplos:** Após seleção de CPU com soquete AM5, todas as placas-mãe com soquete AM4 permanecem visíveis na lista, porém exibidas com indicador de bloqueio. **Restrições:** 1. Componentes incompatíveis **não** devem ser ocultados da listagem. 2. Devem ser mantidos visíveis com indicador visual de bloqueio. 3. O usuário deve poder visualizar o motivo do bloqueio ao interagir com o componente.
Prioridade	Essencial
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 4 – RF-04 – Exibir especificações técnicas dos componentes

Código	RF-04
Grupo	Catálogo de Componentes
Descrição	**Ação:** Exibir. **Objeto:** Especificações técnicas do componente. **Atributos:** soquete, padrão de memória, TDP (W), potência máxima (W) – variáveis conforme a categoria. **Exemplos:** {Ryzen 5 7600, soquete: AM5, TDP: 65 W}; {Kingston DDR5-5600, padrão: DDR5, capacidade: 16 GB}. **Restrições:** 1. As especificações devem ser exibidas para cada componente do catálogo. 2. Os atributos exibidos devem variar conforme a categoria do componente.
Prioridade	Importante
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 5 – RF-05 – Executar propagação de restrições via algoritmo AC-3

Código	RF-05
Grupo	Motor de Inferência (CSP/AC-3)
Descrição	**Ação:** Executar propagação de restrições. **Objeto:** Algoritmo AC-3 sobre o grafo de restrições do CSP. **Atributos:** estado atual das seleções do usuário, domínios das variáveis (CPU, Placa-mãe, RAM, GPU, Fonte), fila de arcos. **Exemplos:** Usuário seleciona CPU com soquete AM5; o AC-3 é invocado e remove do domínio de Placa-mãe todos os modelos cujo soquete seja diferente de AM5. **Restrições:** 1. O AC-3 deve ser invocado a cada seleção de componente realizada pelo usuário. 2. O estado completo das seleções deve ser enviado à API REST a cada chamada (arquitetura *stateless*). 3. O resultado deve retornar os domínios atualizados em tempo real, sem atraso perceptível. 4. A complexidade de pior caso deve ser $O(cd^3)$, conforme definido por Mackworth (1977).
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 6 – RF-06 – Validar compatibilidade de soquete CPU–Placa-mãe

Código	RF-06
Grupo	Motor de Inferência (CSP/AC-3)
Descrição	**Ação:** Validar. **Objeto:** Compatibilidade de soquete entre CPU e Placa-mãe. **Atributos:** soquete da CPU, soquete suportado pela placa-mãe. **Exemplos:** {CPU: AM5, Placa-mãe: AM4} → incompatível – bloqueio acionado; {CPU: AM5, Placa-mãe: AM5} → compatível. **Restrições:** 1. A restrição é binária e absoluta – não admite adaptação de *software* ou configuração. 2. O soquete da CPU deve ser idêntico ao soquete suportado pela placa-mãe. 3. Incompatibilidade detectada deve bloquear o componente e acionar o módulo de explicação (RF-11).
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 7 – RF-07 – Validar padrão de memória RAM–Placa-mãe

Código	RF-07
Grupo	Motor de Inferência (CSP/AC-3)
Descrição	**Ação:** Validar. **Objeto:** Padrão de memória entre módulo RAM e Placa-mãe. **Atributos:** padrão DDR do módulo RAM (DDR4 ou DDR5), padrão DDR suportado pela placa-mãe. **Exemplos:** {RAM: DDR4-3200, Placa-mãe: B650 (somente DDR5)} → incompatível; {RAM: DDR5-5600, Placa-mãe: B650} → compatível. **Restrições:** 1. Os padrões DDR4 e DDR5 são física e eletricamente incompatíveis. 2. O padrão do módulo deve corresponder exatamente ao suportado pela placa-mãe. 3. A restrição é transitiva: depende da placa-mãe, que por sua vez depende da CPU selecionada. 4. Incompatibilidade detectada deve bloquear o componente e acionar o módulo de explicação (RF-12).
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 8 – RF-08 – Validar capacidade energética da fonte (TDP)

Código	RF-08
Grupo	Motor de Inferência (CSP/AC-3)
Descrição	**Ação:** Validar. **Objeto:** Capacidade energética da fonte de alimentação em relação ao consumo dos componentes. **Atributos:** TDP da CPU (W), TDP da GPU (W), potência da fonte (W), margem de segurança recomendada (20–30%). **Exemplos:** {CPU TDP: 65 W, GPU TDP: 170 W, Fonte: 250 W} → bloqueado (235 W × 1,25 = 294 W > 250 W); {Fonte: 550 W} → válido. **Restrições:** 1. A fonte deve suprir a soma dos TDPs de CPU e GPU acrescida de margem de segurança de 20–30%. 2. A restrição é quantitativa (desigualdade), diferentemente das restrições binárias de soquete e memória. 3. Fonte subdimensionada deve ser bloqueada e acionar o módulo de explicação com os valores calculados (RF-13).
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 9 – RF-09 – Propagar restrições incrementalmente a cada seleção

Código	RF-09
Grupo	Motor de Inferência (CSP/AC-3)
Descrição	**Ação:** Propagar restrições e atualizar domínios. **Objeto:** Domínios das variáveis do CSP após cada interação do usuário. **Atributos:** variáveis afetadas, valores removidos dos domínios, arcos reinseridos na fila do AC-3. **Exemplos:** Remoção de valor AM5 do domínio de CPU provoca reinserção do arco (Placa-mãe, CPU) na fila para reverificação dos modelos de placa-mãe ainda disponíveis. **Restrições:** 1. A propagação deve ser incremental a cada interação, nunca por avaliação exaustiva (*brute force*). 2. O motor deve operar até o ponto de estabilidade – quando nenhuma remoção adicional for possível.
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 10 – RF-10 – Exibir justificativa técnica para cada incompatibilidade

Código	RF-10
Grupo	Explicações Educativas (*Explanation Facility*)
Descrição	**Ação:** Exibir justificativa técnica em linguagem natural. **Objeto:** Incompatibilidade detectada pelo AC-3. **Atributos:** restrição violada, componentes envolvidos, princípio técnico subjacente. **Exemplos:** "Este módulo de memória é incompatível: a placa-mãe compatível com o processador selecionado suporta exclusivamente DDR5, e este módulo utiliza o padrão DDR4." **Restrições:** 1. Cada componente bloqueado deve ter uma justificativa exibida ao usuário. 2. A justificativa deve identificar explicitamente qual restrição do CSP foi violada. 3. A linguagem deve ser acessível ao usuário leigo. 4. Nenhum bloqueio pode ocorrer de forma silenciosa (sem explicação ao usuário).
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 11 – RF-11 – Exibir alerta de incompatibilidade de soquete

Código	RF-11
Grupo	Explicações Educativas (*Explanation Facility*)
Descrição	**Ação:** Exibir alerta com explicação. **Objeto:** Incompatibilidade de soquete entre CPU e Placa-mãe. **Atributos:** soquete exigido pela CPU selecionada, soquete oferecido pela placa-mãe bloqueada. **Exemplos:** "Esta placa-mãe utiliza soquete AM4, incompatível com o processador selecionado, que exige soquete AM5." **Restrições:** 1. O alerta deve nomear explicitamente os dois soquetes envolvidos (ex.: AM4 e AM5). 2. Deve ser exibido no momento em que o usuário tenta selecionar o componente incompatível.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 12 – RF-12 – Exibir alerta de incompatibilidade de padrão de memória

Código	RF-12
Grupo	Explicações Educativas (*Explanation Facility*)
Descrição	**Ação:** Exibir alerta com explicação. **Objeto:** Incompatibilidade de padrão de memória entre módulo RAM e Placa-mãe. **Atributos:** padrão DDR suportado pela placa-mãe, padrão DDR do módulo bloqueado. **Exemplos:** "Este módulo utiliza o padrão DDR4. A placa-mãe selecionada suporta exclusivamente DDR5. Os dois padrões são física e eletricamente incompatíveis e não podem ser utilizados juntos." **Restrições:** 1. O alerta deve informar ambos os padrões DDR envolvidos. 2. Deve mencionar a natureza física da incompatibilidade, não apenas a lógica.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 13 – RF-13 – Exibir alerta de fonte de alimentação subdimensionada

Código	RF-13
Grupo	Explicações Educativas (*Explanation Facility*)
Descrição	**Ação:** Exibir alerta com explicação e valores calculados. **Objeto:** Fonte de alimentação com potência insuficiente. **Atributos:** consumo estimado (W), potência mínima recomendada com margem (W), potência da fonte bloqueada (W). **Exemplos:** "Esta fonte possui 250 W. O consumo estimado dos componentes selecionados é de 235 W, exigindo no mínimo 294 W com a margem de segurança de 25%." **Restrições:** 1. O alerta deve exibir o consumo calculado e a potência mínima recomendada. 2. A margem de segurança utilizada no cálculo deve ser informada ao usuário.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 14 – RF-14 – Reinicializar a configuração

Código	RF-14
Grupo	Gerenciamento da Configuração
Descrição	**Ação:** Reinicializar. **Objeto:** Configuração de hardware em andamento. **Atributos:** seleções realizadas, estado do *wizard*. **Exemplos:** Usuário clica em "Reiniciar" na etapa 3; todas as seleções são apagadas e o *wizard* retorna à Etapa 1 (CPU). **Restrições:** 1. A reinicialização deve apagar todas as seleções realizadas. 2. O *wizard* deve retornar ao estado inicial (Etapa 1: CPU). 3. A operação deve estar disponível em qualquer etapa do *wizard*.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 15 – RF-15 – Substituir componente selecionado em etapa anterior

Código	RF-15
Grupo	Gerenciamento da Configuração
Descrição	**Ação:** Substituir. **Objeto:** Componente já selecionado em etapa anterior do *wizard*. **Atributos:** etapa do *wizard*, componente anteriormente selecionado, novo componente escolhido. **Exemplos:** Usuário retorna à Etapa 1 e troca a CPU por um modelo diferente; o sistema reexecuta o AC-3 com a nova CPU e atualiza a disponibilidade dos componentes nas etapas seguintes. **Restrições:** 1. O usuário deve poder retroceder a qualquer etapa anterior do *wizard*. 2. Ao substituir um componente, o sistema deve reexecutar o AC-3 sobre o novo estado completo. 3. Componentes das etapas subsequentes afetados pela substituição devem ser reavaliados automaticamente.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 16 – RF-16 – Exibir resumo da configuração finalizada

Código	RF-16
Grupo	Gerenciamento da Configuração
Descrição	**Ação:** Exibir. **Objeto:** Resumo da configuração de hardware finalizada. **Atributos:** lista dos cinco componentes selecionados, status geral de compatibilidade, consumo total estimado (W). **Exemplos:** Ao concluir a Etapa 5, o sistema exibe: {CPU: Ryzen 5 7600, Placa-mãe: ASUS B650, RAM: Kingston DDR5-5600, GPU: RX 7600, Fonte: 550 W – Status: Configuração compatível}. **Restrições:** 1. O resumo deve ser exibido ao término da seleção de todos os cinco componentes. 2. Deve incluir o status geral de compatibilidade da configuração montada.
Prioridade	Importante
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 17 – RF-17 – Armazenar e consultar base de conhecimento do CSP

Código	RF-17
Grupo	Base de Conhecimento
Descrição	**Ação:** Armazenar e consultar. **Objeto:** Especificações técnicas dos componentes e regras de compatibilidade (conjunto C do CSP). **Atributos:** modelo do componente, categoria, soquete, padrão DDR, TDP (W), restrições binárias entre pares de componentes. **Exemplos:** {CPU: Ryzen 5 7600, soquete: AM5, TDP: 65 W}; restrição: $soquete(CPU, Placa\text{-}mãe)$. **Restrições:** 1. Os dados devem ser persistidos em banco de dados PostgreSQL. 2. As restrições de compatibilidade devem ser extraídas da documentação oficial de AMD, Intel e JEDEC. 3. A base de conhecimento deve ser consultada pela camada de lógica de negócio a cada execução do AC-3.
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

5.2.2 Requisitos Não Funcionais

Fazer uma pequena definição de requisitos não funcionais para introduzir a seção (informar a fonte). Assim, esta seção apresenta os requisitos não funcionais identificados nesse trabalho, apresentados nos Quadros 19 a 32.

Quadro 18 – RF-18 – Popular base de conhecimento via *Database Seeding*

Código	RF-18
Grupo	Base de Conhecimento
Descrição	**Ação:** Popular (*Database Seeding*). **Objeto:** Base de conhecimento inicial da prova de conceito. **Atributos:** conjunto inicial de componentes e restrições para os cenários de validação. **Exemplos:** *Seed* com CPUs AM4 e AM5, placas-mãe B450/B550 (AM4, DDR4) e B650 (AM5, DDR5), módulos DDR4 e DDR5, GPUs e fontes com diferentes potências. **Restrições:** 1. A inserção inicial deve ser realizada via *Database Seeding* no Prisma ORM. 2. O processo de *seeding* deve ser isolado do desenvolvimento de interface administrativa. 3. Os dados devem cobrir ao menos duas plataformas (AM4 e AM5) para viabilizar os cenários de teste.
Prioridade	Desejável
Operação	Entrada
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 19 – RNF-01 – Tempo de resposta da validação

Código	RNF-01
Grupo	Desempenho
Descrição	A execução do algoritmo AC-3 e o retorno dos domínios atualizados via API REST devem ocorrer em milissegundos, garantindo validação em tempo real sem degradação perceptível da interface. **Restrições:** 1. O tempo de resposta da API de validação deve ser inferior a 500 ms para o catálogo da prova de conceito. 2. A resposta deve ser percebida como instantânea pelo usuário durante a interação com o *wizard*.
Prioridade	Essencial
Operação	Processamento
Ator	–

Fonte: Elaborada pelo autor

Quadro 20 – RNF-02 – Ausência de avaliação por força bruta

Código	RNF-02
Grupo	Desempenho
Descrição	O motor de inferência deve operar exclusivamente por propagação de restrições (AC-3), nunca por enumeração exaustiva de combinações, garantindo viabilidade sobre catálogos de tamanho realista. **Restrições:** 1. A complexidade de pior caso do motor deve ser $O(cd^3)$ – nunca $O(d^n)$. 2. A abordagem por força bruta é proibida como estratégia de validação.
Prioridade	Essencial
Operação	Processamento
Ator	–

Fonte: Elaborada pelo autor

Quadro 21 – RNF-03 – Linguagem natural nas explicações de incompatibilidade

Código	RNF-03
Grupo	Usabilidade
Descrição	Todas as justificativas de bloqueio exibidas ao usuário devem ser redigidas em linguagem natural e acessível, sem exigir conhecimento técnico prévio de hardware para sua compreensão. **Restrições:** 1. Termos técnicos utilizados nas explicações devem ser acompanhados de contexto suficiente para compreensão. 2. A explicação deve responder claramente à pergunta "por que este componente está bloqueado?".
Prioridade	Essencial
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 22 – RNF-04 – Interface responsiva

Código	RNF-04
Grupo	Usabilidade
Descrição	A camada de apresentação desenvolvida em Next.js deve ser responsiva, funcionando corretamente em navegadores *desktop* e dispositivos móveis modernos sem perda de funcionalidade. **Restrições:** 1. O *layout* deve adaptar-se a diferentes resoluções de tela. 2. Todas as funcionalidades do *wizard* devem estar acessíveis em dispositivos móveis.
Prioridade	Importante
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 23 – RNF-05 – Acesso sem cadastro ou autenticação

Código	RNF-05
Grupo	Usabilidade
Descrição	O sistema deve ser acessível sem criação de conta, *login* ou qualquer forma de identificação prévia, eliminando barreiras de entrada para o usuário leigo. **Restrições:** 1. Não deve existir tela de *login* ou cadastro para o fluxo principal de configuração. 2. Nenhuma sessão persistente de usuário deve ser exigida para utilização do sistema.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 24 – RNF-06 – Separação estrita entre as três camadas arquiteturais

Código	RNF-06
Grupo	Manutenibilidade
Descrição	O sistema deve respeitar a separação estrita entre camada de dados (PostgreSQL), camada de lógica de negócio (NestJS/AC-3) e camada de apresentação (Next.js), de modo que cada camada possa ser modificada de forma independente das demais. **Restrições:** 1. Alterações na interface não devem requerer modificação do motor de inferência, e vice-versa. 2. A comunicação entre camadas deve ocorrer exclusivamente via API REST.
Prioridade	Essencial
Operação	Processamento
Ator	–

Fonte: Elaborada pelo autor

Quadro 25 – RNF-07 – Motor de inferência expansível para novas restrições

Código	RNF-07
Grupo	Manutenibilidade
Descrição	A implementação do AC-3 deve permitir a adição de novas restrições binárias ao grafo de restrições sem necessidade de refatoração da lógica existente do motor. **Restrições:** 1. Novas restrições devem poder ser cadastradas na base de conhecimento sem alteração de código. 2. O grafo de restrições deve ser construído dinamicamente a partir dos dados persistidos no banco.
Prioridade	Importante
Operação	Processamento
Ator	–

Fonte: Elaborada pelo autor

Quadro 26 – RNF-08 – Base de conhecimento expansível para novas categorias

Código	RNF-08
Grupo	Manutenibilidade
Descrição	O modelo de dados deve permitir a inclusão futura de novas categorias de componentes além das cinco iniciais (CPU, Placa-mãe, RAM, GPU e Fonte) sem quebra do esquema existente. **Restrições:** 1. A estrutura da base de conhecimento não deve ser *hardcoded* para exatamente cinco categorias. 2. O motor de inferência deve operar genericamente sobre qualquer grafo de restrições fornecido.
Prioridade	Desejável
Operação	Processamento
Ator	–

Fonte: Elaborada pelo autor

Quadro 27 – RNF-09 – Restrições derivadas de documentação oficial dos fabricantes

Código	RNF-09
Grupo	Confiabilidade
Descrição	Todas as restrições de compatibilidade cadastradas na base de conhecimento devem ser extraídas e referenciadas à documentação técnica oficial de AMD, Intel e JEDEC, garantindo a fidelidade técnica das regras do motor de inferência. **Restrições:** 1. Cada regra de compatibilidade deve ter sua fonte documental identificável. 2. Regras sem respaldo em documentação oficial de fabricante não devem ser incluídas na base de conhecimento.
Prioridade	Essencial
Operação	Processamento
Ator	–

Fonte: Elaborada pelo autor

Quadro 28 – RNF-10 – Determinismo do motor de inferência

Código	RNF-10
Grupo	Confiabilidade
Descrição	Para o mesmo estado de seleções de entrada, o sistema deve sempre produzir o mesmo conjunto de componentes válidos e as mesmas justificativas de bloqueio, sem qualquer variação não determinística. **Restrições:** 1. O motor de inferência deve ser estritamente determinístico: mesma entrada produz mesma saída. 2. Não deve haver comportamento aleatório, probabilístico ou dependente de estado externo volátil.
Prioridade	Essencial
Operação	Processamento
Ator	–

Fonte: Elaborada pelo autor

Quadro 29 – RNF-11 – API *stateless* (sem estado de sessão no servidor)

Código	RNF-11
Grupo	Escalabilidade e Operação
Descrição	A API REST deve operar sem manter estado de sessão entre requisições, recebendo o estado completo das seleções do usuário a cada chamada, em conformidade com o princípio *stateless* definido por Fielding (2000). **Restrições:** 1. Nenhum dado de sessão de usuário deve ser armazenado no servidor entre requisições. 2. Cada requisição deve ser autossuficiente e conter todo o contexto necessário para o processamento.
Prioridade	Essencial
Operação	Processamento
Ator	–

Fonte: Elaborada pelo autor

Quadro 30 – RNF-12 – Ambiente de banco de dados reproduzível via Docker

Código	RNF-12
Grupo	Escalabilidade e Operação
Descrição	O provisionamento do PostgreSQL deve ser realizado via Docker, garantindo consistência entre os ambientes de desenvolvimento e produção de forma declarativa e reproduzível. **Restrições:** 1. O ambiente do banco de dados deve ser definido por arquivo de configuração Docker (`docker-compose`). 2. A instalação direta do PostgreSQL no sistema operacional do desenvolvedor não deve ser necessária.
Prioridade	Importante
Operação	Processamento
Ator	–

Fonte: Elaborada pelo autor

Quadro 31 – RNF-13 – Rastreabilidade de cada bloqueio até a restrição violada

Código	RNF-13
Grupo	Transparência (XAI)
Descrição	O sistema deve ser capaz de identificar e comunicar, para cada componente bloqueado, qual restrição específica do CSP foi violada, alinhando-se ao princípio de transparência da Inteligência Artificial Explicável (**BarredoArrieta2020**). **Restrições:** 1. Cada bloqueio deve ser rastreável até a aresta do grafo de restrições que o originou. 2. O sistema deve ser auditável: o raciocínio do motor de inferência deve poder ser inspecionado e verificado.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

5.2.3 Restrições, Suposições e Dependências

O sistema possui as seguintes dependências e restrições de desenvolvimento:

- As regras de compatibilidade devem ser extraídas exclusivamente da documentação técnica oficial dos fabricantes AMD, Intel e JEDEC;
- O banco de dados PostgreSQL deve estar provisionado via Docker antes da execução do sistema;
- O catálogo inicial de componentes deve ser populado via *Database Seeding* no Prisma ORM antes dos testes de validação.

Quadro 32 – RNF-14 – Ausência de bloqueios silenciosos

Código	RNF-14
Grupo	Transparência (XAI)
Descrição	Nenhum bloqueio de componente deve ocorrer sem exibição de justificativa ao usuário. O sistema não deve implementar filtragem silenciosa, diferenciando-se explicitamente de plataformas como PC Part Picker e Meupc.net, que sinalizam incompatibilidades sem explicá-las (Clancey, 1983). **Restrições:** 1. A ausência de justificativa para qualquer bloqueio constitui falha de requisito. 2. A *explanation facility* deve ser tratada como característica nuclear do sistema, não como funcionalidade acessória.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

5.2.4 Requisitos Adiados

Os seguintes requisitos foram identificados, mas serão desenvolvidos em momento posterior:

- Interface administrativa para cadastro e edição de componentes via *front-end* (CRUD completo);
- Ampliação do catálogo para além das cinco categorias iniciais (ex.: cooler, SSD, gabinete);
- Geração de link compartilhável com a configuração de hardware finalizada;
- Estratégia de testes automatizados (unitários do AC-3 e de integração da API REST).

5.3 Casos de Uso

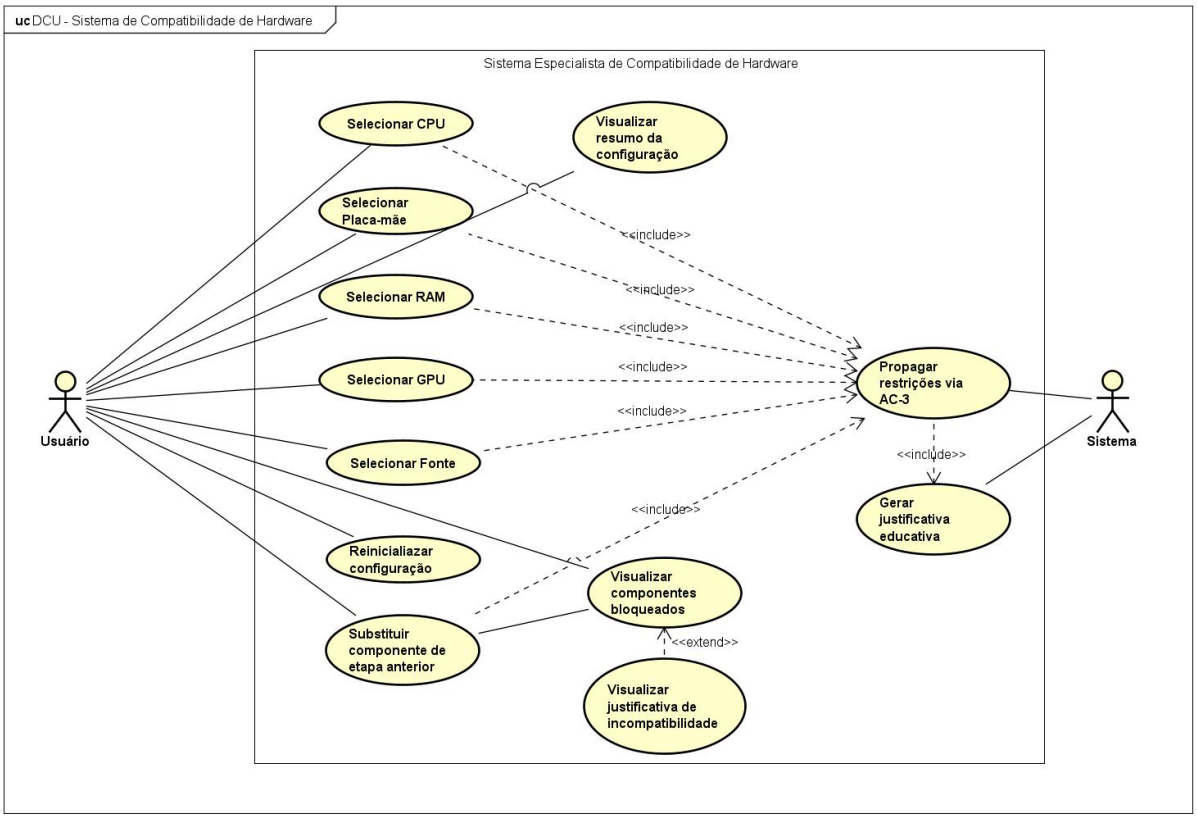
5.4 Casos de Uso

Forneça uma breve definição de casos de uso (com referência da fonte) e informe que os casos de uso identificados neste trabalho estão aqui apresentados.

5.4.1 Diagrama de Casos de Uso

Forneça uma breve definição de diagrama de casos de uso (referência da fonte). O diagrama de casos de uso elaborado neste trabalho está apresentado na Figura 4.

Figura 4 – Diagrama de Casos de Uso



Fonte: Elaborada pelo autor

5.4.2 Especificação dos Casos de Uso

Uma especificação de caso de uso fornece detalhe textual para um caso de uso. Uma estrutura de tópicos de amostra de uma especificação de caso de uso é fornecida. É possível reutilizar e modificar a estrutura de tópicos conforme necessário em um documento de especificação de caso de uso. A Tabela 1 apresenta um modelo para especificação de casos de uso (extraído de IBM-Especificação de Casos de Uso).

Tabela 1 – Modelo para Especificação de Casos de Uso

Item	**Descrição**
Nome do Caso de Uso	Declara o nome do caso de uso. Geralmente, o nome expressa o objetivo ou resultado observável do caso de uso, como "Sacar Dinheiro"no caso de um caixa eletrônico.
Descrição Resumida	Descreve a função e o objetivo do caso de uso.
Fluxo de Eventos	Apresenta o fluxo básico e os fluxos alternativos. O fluxo de eventos descreve o comportamento do sistema; ele não descreve como o sistema funciona, os detalhes da apresentação ou os detalhes da interface com o usuário. Se forem trocadas informações, o caso de uso deverá ser específico sobre o que será transmitido de um lado para outro. Por exemplo, em vez de descrever uma ação como "o agente insere informações do cliente", indique que "o agente insere o nome e o endereço do cliente".
Fluxo Básico	Descreve o comportamento principal ideal do sistema.
Fluxos Alternativos	Descreve exceções ou desvios do fluxo básico, como o comportamento do sistema quando o agente insere um ID de usuário incorreto e a autenticação do usuário falha.
Requisitos Especiais	Requisitos não-funcionais que são específicos para um caso de uso mas que não são especificados no texto do fluxo do caso de uso de eventos. Exemplos de requisitos especiais incluem estes fatores: requisitos legais e regulamentares; padrões de aplicativo; atributos de qualidade do sistema, incluindo usabilidade, confiabilidade, desempenho e capacidade de suporte; sistemas operacionais e ambientes; requisitos de compatibilidade e restrições de design.
Condições Prévias	Um estado do sistema que deve estar presente antes de um caso de uso ser iniciado.
Pós-Condições	Uma lista de estados possíveis para o sistema imediatamente após a conclusão de um caso de uso.
Pontos de Extensão	Um ponto no fluxo de eventos do caso de uso em que outro caso de uso é referenciado.

Fonte: Extraído de IBM-Especificação de Casos de Uso

Elaborar as especificações para cada caso de uso, quando for necessário, como por exemplo: A especificação do caso de uso Finalizar Compra é

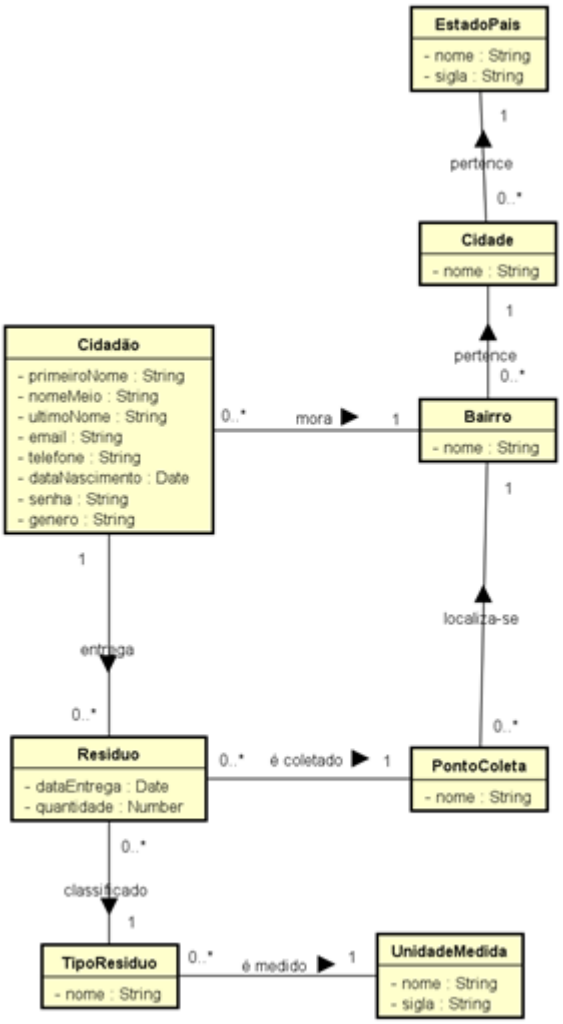
5.5 Modelo de Domínio

Fazer uma pequena definição de modelo de domínio para introduzir essa seção. Terminar o parágrafo informando qual figura apresenta o modelo de domínio. Elabore o diagrama de classes somente com os atributos, para auxiliar no entendimento do domínio da aplicação, conforme Figura 5.

De acordo com **braga2008algoritmos**, o modelo de domínio é ..... . A Figura 5 apresenta o modelo de domínio desenvolvido neste trabalho.

Caso de Uso Finalizar Comprar
Referências RF02
Descrição Geral O caso de uso inicia-se quando o cliente deseja efetuar compra dos produtos que estão inseridos no carrinho de compras.
Atores Cliente
Pré-Condições Cliente logado no sistema, produtos já inseridos no carrinho de compras.
Garantia de Sucesso (Pós-Condições Pedido fechado, compra efetuada, sistema aguardando confirmação de pagamento.
Requisitos Especiais
Fluxo Básico 1. Cliente deseja finalizar compra, sistema solicita que informe a forma de pagamento e de entrega; 2. Cliente deseja efetuar pagamento em forma de cartão de crédito/débito; 3. Sistema solicita informações do cartão do cliente; 4. Sistema faz validação das informações; 5. Sistema gera o número do pedido; 6. Compra finalizada com sucesso.
Fluxo Alternativo 1. Cliente deseja efetuar pagamento através de boleto bancário 1.1 Sistema gera boleto para o cliente; 1.2 Retorne ao passo 5.

Figura 5 – Modelo de Domínio



Fonte: Elaborada pelo autor

5.6 Diagrama de Objetos

Fazer uma pequena definição de diagrama de objetos para introduzir essa seção. Terminar o parágrafo informando qual figura apresenta o diagrama de objetos. Elabore o diagrama de objetos para cada classe identificada no modelo de domínio.

5.7 Diagrama de Classes de Análise

Fazer uma pequena definição de diagrama de classes para introduzir essa seção. Terminar o parágrafo informando qual figura apresenta o diagrama de classes. É uma evolução do modelo de domínio, elabore o diagrama de classes com os atributos e métodos.

5.8 Diagrama de Atividades

Um diagrama de atividade ilustra a natureza dinâmica de um sistema pela modelagem do fluxo de controle de atividade à atividade. Uma atividade representa uma operação em alguma classe no sistema que resulta em uma mudança no estado do sistema.

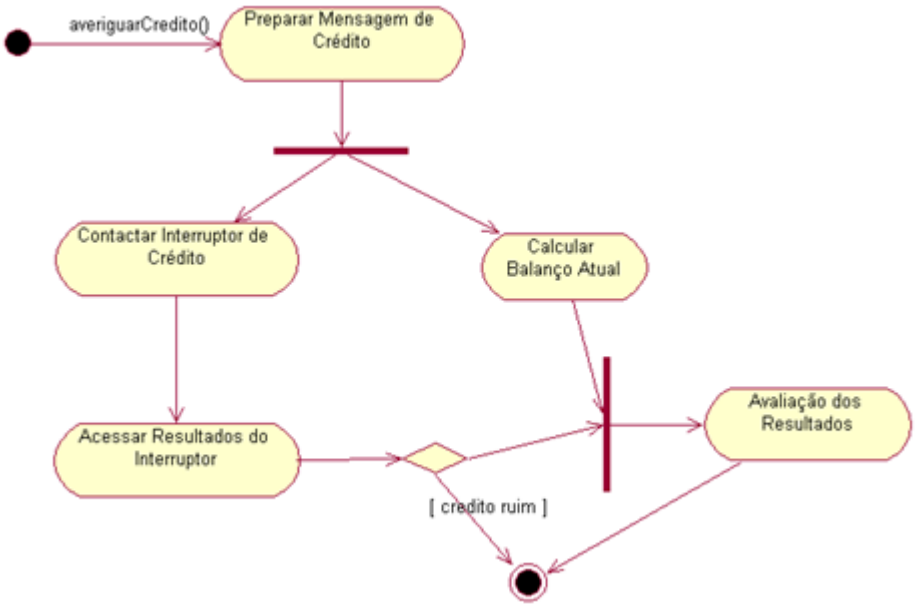
Tipicamente, diagramas de atividades são usados para modelar fluxos de processos, processos de negócios ou operações internas. o diagrama de atividades é similar a uma máquina de estados, mas tem um propósito diferente, o qual envolve capturar ações e seus resultados em termos de mudanças do estado do objeto.

O diagrama de atividades é representado por um gráfico de atividades que mostram o fluxo de uma atividade para outra. Esse fluxo é mostrado através de transições, que são setas direcionadas, mostrando o caminho entre os estados de atividade (ação).

A Figura 6 mostra o diagrama de atividades para a operação averiguarCredito na classe Pedido da Virtual LTDA. Note que a atividade "Preparar Mensagem de Credito" define o que fazer, mas não como fazer (RIBEIRO, 2021).

Um diagrama de atividades é normalmente composto pelos seguintes elementos: atividades (ações), estados de atividade (ação), transição, fluxo de objeto, estado inicial, estado final, branching, sincronização, raias.

Figura 6 – Diagrama de Atividades

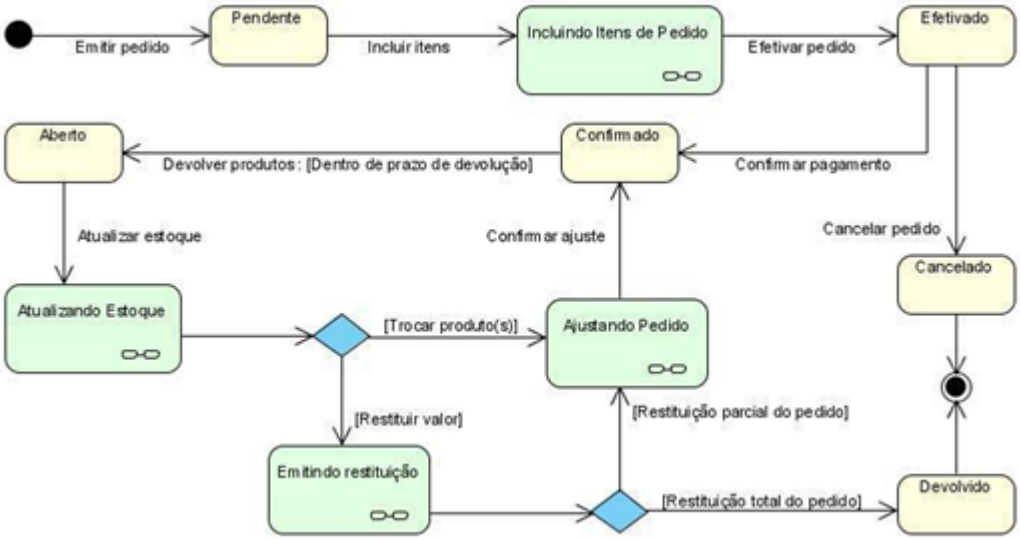


Fonte: Elaborada pelo autor

5.9 Diagrama de Estados

Fazer uma introdução mais ou menos como o parágrafo a seguir. Um diagrama de estados mostra os possíveis estados de um objeto e as transações responsáveis pelas suas mudanças de estado, conforme exemplo apresentado na Figura 7.

Figura 7 – Diagrama de Estados



Fonte: Elaborada pelo autor

6 PROJETO DE SOFTWARE

Parte principal do texto, que contém a exposição ordenada e pormenorizada do assunto. Divide-se em seções e subseções, que variam em função da abordagem do tema e do método.

6.1 Projeto de Interface

Apresentar o protótipo das interfaces do sistema. Podem ser construídas utilizando qualquer ferramenta. Apresente os padrões utilizados.

6.2 Projeto de Dados

Informar nesta seção qual o Banco de Dados a ser utilizado, qual a IDE utilizada para manipulação do Banco de Dados, assim como descrever quais os tipos de dados aceitos no Banco de Dados.

6.2.1 Mapeamento Objeto-Relacional

Uma vez que foi elaborada uma modelagem orientada a objetos, utilizando o diagrama de classes UML, e o banco de dados a ser utilizado é relacional, deve-se identificar as relações. Assim, a partir dos requisitos e do modelo de domínio enumerar as relações que devem formar o banco de dados.

Regras para realizar o mapeamento: (Helen, colocar as regras aqui para que os alunos escrevam)

- Item 1;
- Item 2;
- Item n.

6.2.2 Estrutura das Tabelas no Banco de Dados

Explicar qual o padrão adotado para o nome dos objetos de banco de dados, como por nome da chave primária, chave estrangeira, das constraints e das chaves únicas. Por exemplo: foi convencionado que o nome dos objetos devem obedecer o que está definido na Tabela 2.

Tabela 2 – Convenção para Nome dos Objetos no Banco de Dados

Objeto	**Padrão Adotado**
Chave Primária	NomeDaTabela_PK
Chave Estrangeira	NomeDaTabela_NomeDaTabelaEstrangeira_FK_nn, onde nn é a sequência de ocorrência do par NomeDaTabela e NomeDaTabelaEstrangeira
Check	NomeDaTabela_CK_nn, onde nn é a sequência de checks da tabela
Chave Única	NomeDaTabela_UK_nn, onde nn é a sequência de chave única da tabela

Fonte: Elaborada pelo autor

Para melhor compreensão, as tabelas do banco de dados propostas neste trabalho estão consolidadas na Tabela 3, as quais estão individualmente detalhadas.

Tabela 3 – Tabelas Identificadas neste Trabalho

Tabela do Banco de Dados	**Tabela no Documento**
Estado	Tabela 4
Cidade	Tabela 5
NomeDaTabelann	Tabela 6

Fonte: Elaborada pelo autor

Preencher o quadro a seguir para cada uma das relações identificadas no item Mapeamento OO-Relacional, que deve refletir as características das tabelas a serem criadas no banco de dados. Para melhor compreensão sobre qual a estrutura física das tabelas, seguem dois exemplos: a Tabela 6 é um exemplo da estrutura de dados Estado e a Tabela 6 é um exemplo da estrutura de dados Cidade.

Tabela 4 – Estado

Campo	**Tipo de Dado**	**Obrigatório?**	**Chave Primária?**	**Tabela**	**Campo**	**Grupo**
id	**Number**	**X**	**X**			
nome	**Varchar2(100)**	**X**				**1**
sigla	**Varchar2(2)**	**X**				**2**

Fonte: Elaborada pelo autor

INSTRUÇÃO: Grupo é a chave única que você deseja criar (UK_01, UK_02) Ordem é a sequência dos campos que devem compor a chave única, quando existir mais de um campo na sua chave única

Tabela 5 – Cidade

Campo	Tipo de Dado	Obrigatório?	Chave Primária?	Tabela	Campo	Grupo
id	**Number**	**X**	**X**			
nome	**Varchar2(200)**	**X**				**1**
idEstado	**Number**	**X**		**Estado**	**id**	**1**
latitude	**Number**	**X**				**2**
longitude	**Number**	**X**				**2**

Fonte: Elaborada pelo autor

Tabela 6 – NomeDaTabelaNN

Campo	Tipo de Dado	Obrigatório?	Chave Primária?	Tabela	Campo	Grupo

Fonte: Elaborada pelo autor

6.2.3 Diagrama de Pacotes

Escreva aqui.

6.2.4 Diagrama de Classes de Projeto

Apresentar as classes de projeto identificadas. Alguns materiais para leitura extra sobre classes de projeto podem ser lidos em: `http://www.ic.uff.br/~anselmo/cursos/ProjSoft/apresentacoes/Projeto%20detalhado%20-%20Classes.pdf` `https://edisciplinas.usp.br/pluginfile.php/383727/mod_resource/content/2/Aula07_DiagramaDeClasse.pdf` `http://www.dsc.ufcg.edu.br/~jacques/cursos/apoo/html/proj1/proj8.htm`

6.3 Projeto Procedimental

Projetar Software é o processo de aplicar várias técnicas e princípios com o propósito de se definir um dispositivo, processo ou sistema, com detalhes suficientes para permitir sua realização física (Taylor-59).

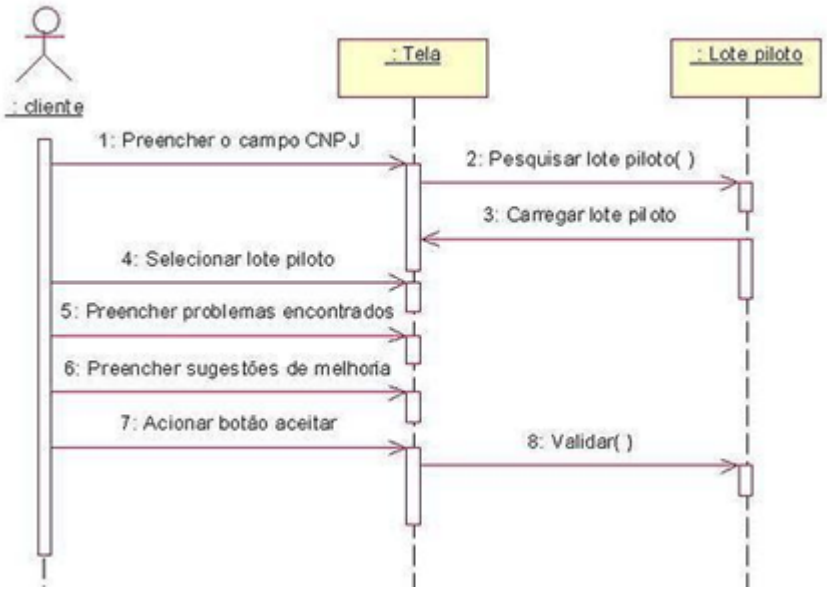
O Projeto de software é o núcleo técnico da Engenharia de Software. É a única maneira de se traduzir "com precisão", os requisitos do usuário para um produto ou sistema acabado. Meta: Traduzir requisitos numa representação de software (Portella).

6.3.1 Diagrama de Sequência

Os diagramas de sequência modelam as interações entre objetos em um único caso de uso e ilustram como as diferentes partes de um sistema interagem entre si para realizar

uma função e a ordem em que as interações ocorrem quando um caso de uso é executado (Creately, s.d.), conforme apresentado na Figura 8.

Figura 8 – Diagrama de Sequência



Fonte: Elaborada pelo autor

6.3.2 Diagrama de Atividades

Havendo necessidade de detalhamento de algum procedimento, este pode ser apresentado na forma de um diagrama de atividades o qual deve ser inserido nessa seção. Podem ser elaborados quantos diagramas de atividades se fizerem necessários.

6.3.3 Diagrama de Estados

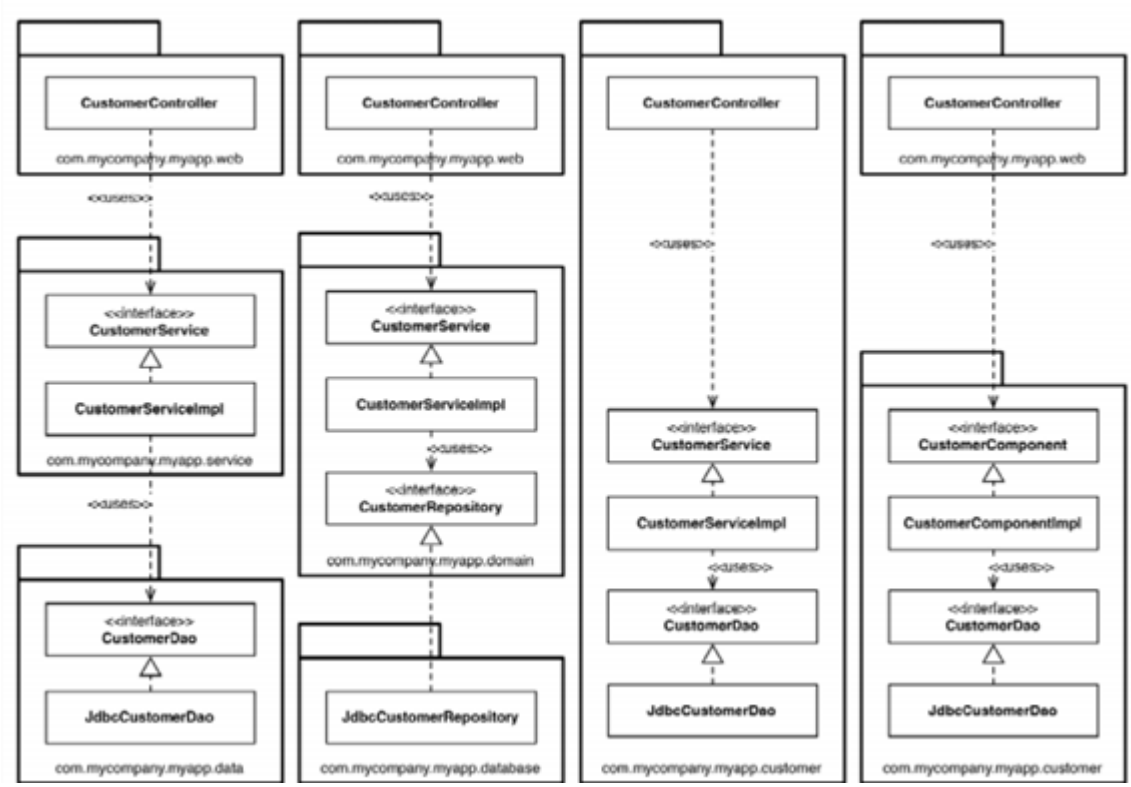
Sendo necessária a elaboração do diagrama de estados na fase de projeto, os novos diagramas devem ser apresentados nessa seção.

6.4 Projeto Arquitetural

O projeto arquitetural precede a etapa de construção da obra. O projeto arquitetural determina as partes de uma construção e como estas devem interagir. A arquitetura garante a unidade da obra, ou seja, a consistência entre as suas partes (Vergilio).

Ver algumas definições em (Silva), sendo que um exemplo está apresentado na Figura 9.

Figura 9 – Diferentes Detalhamentos dos Serviços Relacionados ao Cliente



Fonte: Elaborada pelo autor

7 IMPLEMENTAÇÃO

Descrever nesse capítulo as características do ambiente de desenvolvimento, servidor de aplicação, servidor de dados, pacotes utilizados etc etc etc.

8 TESTE

Descrever nesse capítulo quais e como foram os testes realizados. Os testes podem ser apresentados na forma de casos de teste. Um caso de teste consiste em conjunto de detalhes necessários para se realizar um teste de software.

A seguir encontra-se um modelo para especificação de um caso de teste (CEDRO, 2021):

Título O título do caso de teste deverá ser sucinto, simples e autoexplicativo com informações para que o Analista de Teste saiba a validação a qual o teste se propõe. Exemplos: • Validar upload de arquivo; • Validar cadastro de usuário com perfil administrador; • Validar envio de ordem de compra.
Objetivo O objetivo do caso de teste é descrever o que será executado, fornecendo uma visão geral do teste que será realizado. Exemplos: • Verificar se realiza o upload do arquivo com as extensões permitidas; • Verificar se o cadastro é efetivado após preencher as informações corretamente; • Verificar se a ordem de compra é enviada informando o ativo, quantidade e preço.
Pré-Condição São condições necessárias para que o caso de teste consiga ser executado. Evitar que não tenha alguma informação necessária (Exemplo: solicitar a edição de um usuário em específico e na pré-condição não informar que o usuário deve estar cadastrado). Exemplos: • Usuário cadastrado e autenticado no sistema; • Ordem de compra enviada e executada; • Usuário com perfil Administrador.

Passos Os passos são necessários para descrever todas as ações que o analista deve seguir durante a execução para chegar ao resultado esperado. Devendo iniciar com um verbo infinitivo (acessar, preencher, clicar, verificar) ou imperativo (acesse, preencha, clique, verifique). Exemplos: • Acessar a tela Negociação > Boleta; • Clicar no botão "Entrar"; • Verificar se a edição foi salva no banco de dados; • Preencher os campos do cadastro.
Resultados Esperados Descrever o comportamento esperado do sistema após executar os passos detalhados. Informar os verbos no presente (valida, apresenta, recupera, retorna). Evitar frases como "O sistema **deve** retornar a mensagem. . . ", prefira usar "O sistema retorna a mensagem. . . " para não deixar nenhuma dúvida do resultado esperado. Exemplos: • Sistema apresenta a tela de edição com os campos preenchidos.; • A ordem é enviada e executada com o preço informado; • O cadastro é salvo no banco de dados.

Os casos de teste podem ser especificados usando uma ferramenta de software. Caso este seja o caso, esta seção deve apresentar qual a ferramenta e foi utilizada no desenvolvimento deste trabalho. A Figura 10 apresenta um exemplo de caso de teste especificado usando a ferramenta Testlink.

Figura 10 – Exemplo de Caso de Teste Elaborado na Ferramenta Testlink

Caso de Teste

CT-AUT-140:Validar cadastro de cliente

Versão 1

Objetivo do Teste:

Verificar se realiza o cadastro do cliente informando nome, CPF e telefone.

Pré-condições

1. Usuário cadastrado e autenticado no Portal ABC;
2. Usuário com perfil Administrador;
3. Possuir CPF válido.

#	Ações do Passo	Resultados Esperados:
1	1 - Acessar a tela de cadastro de cliente no Portal ABC: Menu principal > Cadastros > Cliente; 2 - Preencher os campos com dados válidos: • Nome • CPF • Telefone 3 - Clicar em Salvar; 4 - Verifique se o cadastro do cliente foi salvo no Banco de dados.	1 - Sistema exibe a tela de cadastro de cliente com os campos vazios; 2 - Após salvar o cadastro exibe a mensagem de sucesso: "Cliente cadastrado."; 3 - O registro do cliente é salvo no Banco de dados.

Fonte: Elaborada pelo autor

9 IMPLANTAÇÃO

Em qual servidor o sistema está implantado, em qual servidor está a aplicação e em qual servidor está o banco de dados, se foi feito registro de domínio, como deve ser feita a implantação e configuração do sistema, etc etc etc.

Caso o sistema esteja em execução em um ou mais clientes, apresentar nesse capítulo como foi a implantação. Sendo necessário, podem ser usados diagramas UML, como o diagrama de atividades, para apresentar as tarefas de implantação.

10 RESULTADOS E DISCUSSÃO

Texto dos resultados.

11 CONCLUSÕES/CONCLUSÕES PARCIAIS

Texto das conclusões.

Obs: Este capítulo deve ser intitulado “Conclusões Parciais” em trabalhos de graduação para a Validação de Projeto de TCC. Na Avaliação Final de TCC o nome do capítulo deve ser “Conclusões”.

12 CRONOGRAMA

Segue abaixo o cronograma das atividades que serão executadas até a Avaliação Final de TCC.

Obs: Para facilitar, crie o cronograma usando o modelo do Word contido no projeto (imagens/templateCronograma.docx), ou qualquer outro *software*, salve a imagem e atualize o arquivo imagens/cronograma.png.

		Meses											
		JAN	FEV	MAR	ABR	MAI	JUN	JUL	AGO	SET	OUT	NOV	DEZ
Atividades	1	X	X										
	2		X	X	X								
	3			X	X	X							
	4			X	X								
	5	X	X	X	X	X	X	X	X	X	X	X	X

1. Descrição da atividade 1;
2. Descrição da atividade 2;
3. Descrição da atividade 3;
4. Descrição da atividade 4;
5. Descrição da atividade 5.

Obs: Este capítulo deve ser elaborado e estar contido em trabalhos de graduação para a Validação de Projeto de TCC. Na Avaliação Final de TCC este capítulo não deve existir, visto que não haverá atividades após a Avaliação Final.

REFERÊNCIAS

ADVANCED MICRO DEVICES. **Meet the New AMD Socket AM5 Platform**. [*S. l.: s. n.*], 2022. AMD Partner Insights. Disponível em: `https://www.amd.com/en/partner/articles/socket-am5-change-game.html`. Citado nas pp. 15, 16, 18, 21.

ANDERSON, John R. *et al.* Cognitive Tutors: Lessons Learned. **The Journal of the Learning Sciences**, v. 4, n. 2, p. 167–207, 1995. DOI: `10.1207/s15327809jls0402_2`. Citado na p. 30.

BARREDO ARRIETA, Alejandro *et al.* Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI. **Information Fusion**, v. 58, p. 82–115, 2020. DOI: `10.1016/j.inffus.2019.12.012`. Citado nas pp. 29, 35.

CLANCEY, William J. The Epistemology of a Rule-Based Expert System: A Framework for Explanation. **Artificial Intelligence**, v. 20, n. 3, p. 215–251, 1983. DOI: `10.1016/0004-3702(83)90008-5`. Citado nas pp. 15, 16, 28–31, 34, 35, 37, 42, 61.

CORMEN, Thomas H. *et al.* **Introduction to Algorithms**. 2. ed. Cambridge: The MIT Press, 2001. Citado nas pp. 21, 23, 24.

DECHTER, Rina. **Constraint Processing**. San Francisco: Morgan Kaufmann, 2003. Citado nas pp. 20, 26.

DIESTEL, Reinhard. **Graph Theory**. 5. ed. Berlin: Springer, 2017. Disponível em: `https://diestel-graph-theory.com`. Citado na p. 21.

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. 2000. Tese (Doutorado) – University of California, Irvine. Disponível em: `https://ics.uci.edu/~fielding/pubs/dissertation/top.htm`. Citado nas pp. 32, 34, 38, 59.

FONTELLES, Mauro José *et al.* Metodologia da pesquisa científica: Diretrizes para a elaboração de um protocolo de pesquisa. **Revista Paraense de Medicina**, v. 23, n. 3, p. 1–8, 2009. Disponível em: `http://files.bvs.br/upload/S/0101-5907/2009/v23n3/a1967.pdf`. Citado na p. 37.

FOWLER, Martin. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002. Citado nas pp. 31, 32, 34, 38, 39.

GERHARDT, Tatiana Engel; SILVEIRA, Denise Tolfo. **Métodos de Pesquisa**. Porto Alegre: Editora da UFRGS, 2009. Citado na p. 38.

INTEL CORPORATION. **ATX Specification Version 3.0**. Santa Clara, 2022. Disponível em: `https://edc.intel.com`. Citado na p. 19.

INTEL CORPORATION. **LGA1851 and LGA1700 Socket Incompatibility**. [*S. l.: s. n.*], 2023. Intel Support Knowledge Base. Article ID: 000099723. Disponível em: `https://www.intel.com/content/www/us/en/support/articles/000099723/processors.html`. Citado nas pp. 15, 16, 18.

JACKSON, Peter. **Introduction to Expert Systems**. 3. ed. Boston: Addison-Wesley, 1998. Citado nas pp. 28, 29, 31.

JEDEC SOLID STATE TECHNOLOGY ASSOCIATION. **JESD79-5B: DDR5 SDRAM Standard**. Arlington, 2020. Disponível em: `https://www.jedec.org`. Citado nas pp. 15, 16, 18, 21.

JUKNA, Stasys *et al.* On P versus NP ∩ co-NP for Decision Trees and Read-Once Branching Programs. **Computational Complexity**, v. 8, n. 4, p. 357–370, 1999. DOI: `10.1007/s000370050005`. Citado na p. 24.

KUMAR, Vipin. Algorithms for Constraint-Satisfaction Problems: A Survey. **AI Magazine**, v. 13, n. 1, p. 32–44, 1992. DOI: `10.1609/aimag.v13i1.976`. Citado nas pp. 21, 25, 26.

MACKWORTH, Alan K. Consistency in Networks of Relations. **Artificial Intelligence**, v. 8, n. 1, p. 99–118, 1977. DOI: `10.1016/0004-3702(77)90007-8`. Citado nas pp. 16, 26, 27, 37, 38, 41, 47.

MEUPC.NET. **Meupc.net: Monte seu PC**. [*S. l.: s. n.*], 2024. Plataforma web. Disponível em: `https://www.meupc.net`. Citado nas pp. 15, 16, 34.

MITTAL, Sanjay; FRAYMAN, Felix. Towards a Generic Model of Configuration Tasks. *In:* PROCEEDINGS of the 11th International Joint Conference on Artificial Intelligence (IJCAI-89). Detroit: Morgan Kaufmann, 1989. p. 1395–1401. Citado na p. 35.

NWANA, Hyacinth S. An Introduction to Intelligent Tutoring Systems. **The Knowledge Engineering Review**, v. 5, n. 4, p. 251–276, 1990. DOI: `10.1017/S0269888900005014`. Citado na p. 35.

PAPADIMITRIOU, Christos H.; YANNAKAKIS, Mihalis. On Limited Nondeterminism and the Complexity of the V-C Dimension. **Journal of Computer and System Sciences**, v. 53, n. 2, p. 161–170, 1996. DOI: `10.1006/jcss.1996.0058`. Citado na p. 23.

PCPARTPICKER LLC. **PC Part Picker: System Builder**. [*S. l.: s. n.*], 2024. Plataforma web. Disponível em: `https://pcpartpicker.com`. Citado nas pp. 15, 16, 34.

RICHARDSON, Leonard; RUBY, Sam. **RESTful Web Services**. Sebastopol: O'Reilly Media, 2007. Citado na p. 32.

RUSSELL, Stuart; NORVIG, Peter. **Artificial Intelligence: A Modern Approach**. 3. ed. Upper Saddle River: Prentice Hall, 2010. Citado nas pp. 16, 19, 21, 22, 24–28, 37, 41.

SABIN, Daniel; WEIGEL, Rainer. Product Configuration Frameworks: A Survey. **IEEE Intelligent Systems**, v. 13, n. 4, p. 42–49, 1998. DOI: `10.1109/5254.708432`. Citado nas pp. 34, 35.

SIPSER, Michael. **Introduction to the Theory of Computation**. 3. ed. Stamford: Cengage Learning, 2012. Citado nas pp. 16, 23, 24.

WANG, Cixiao; WU, Feng. An Expert System Approach to Support Blended Learning in Context-Aware Environment. *In:* CHEUNG, Simon K. S. *et al.* (ed.). **Blended Learning: Enhancing Learning Success. Proceedings of the 11th International Conference on Blended Learning (ICBL 2018)**. Cham: Springer, 2018. v. 10949. (Lecture Notes in Computer Science), p. 24–35. DOI: `10.1007/978-3-319-94505-7_3`. Citado nas pp. 35, 36.

WOOLF, Beverly Park. **Building Intelligent Interactive Tutors: Student-Centered Strategies for Revolutionizing E-Learning**. San Francisco: Morgan Kaufmann, 2008. Citado na p. 30.